

KOGI

Independent Worker Operating System

Interconnected Master Distributed Spreadsheet System

Software Design Document

KPMS · KPSS · KPRG · KPVW · KPCM · KPCL · KPEX · KLNK · KPID · KSPC · KWSP · KNSPC
Spaces · Workspaces · Namespaces · Multi-Identity · Link Network · Forest & Tree Architecture

Attribute	Value
Document Status	Active Design — v1.0, March 2026
Classification	Confidential — Kogi Platform Internal
Module Codes	KPMS · KPSS · KPRG · KPVW · KPCM · KPCL · KPEX · KLNK · KPID · KSPC · KWSP · KNSPC
Core Languages	Rust (Substrate) · Go (Services) · Scala 3 (kogi-engine) · TypeScript (UI) · SQL (PostgreSQL)
Companion SDDs	Portfolio System SDD v2.1 · kogi-bank SDD · kogi-community SDD v2.0 · kogi-game SDD · kogi-marketplace SDD · kogi-rms SDD · kogi-wms SDD · KMDSS SDD v1.0
Primary Authors	Kogi Platform Team
Target Audience	Platform engineers, system architects, product managers, contributors

Table of Contents

1. Document Purpose & Scope

This Software Design Document (SDD) specifies the complete technical design of the Kogi Interconnected Master Distributed Spreadsheet System (KIMDSS) — the central data substrate, computation engine, distributed consistency layer, identity system, spatial organization architecture, and inter-portfolio link network of the Kogi Independent Worker Operating System (Kogi IW-OS).

The KIMDSS is simultaneously:

- A universal, event-sourced, CRDT-distributed spreadsheet where every entity in the Kogi ecosystem is a row, every property is a column, and every perspective is a sheet
- A spatial organization system of Spaces, Workspaces, and Namespaces that structures how portfolios, identities, and organizations are grouped, scoped, and discovered
- A multi-identity, multi-account, multi-profile system enabling a single Sovereign Entity to maintain multiple @handles, partition their data across identities, and control visibility with fine-grained masks
- A global inter-portfolio link network (KLNK) mapping every connection between portfolios, components, identities, and federation nodes as a traversable graph of Forests and Trees
- A computation platform running 20+ analytical models producing health scores, risk scores, income projections, and hundreds of AI-derived signals inline as computed columns
- A collaborative editing platform with CRDT-based conflict resolution, real-time multi-user editing, and contribution attribution for shared and crowdresourced portfolios

1.1 Scope Boundaries

In Scope	Out of Scope
Core data model: PortfolioComponent, PortfolioRow, all column types, all sheet definitions	Business logic specific to individual kogi-* application modules (covered in per-module SDDs)
Spaces, Workspaces, and Namespaces architecture (KSPC · KWSP · KNSPC)	UI/UX wireframes and front-end component implementation
Multi-identity, multi-account, multi-profile system (KPID)	Specific kogi-engine ML model implementations (covered in kogi-engine SDD)
KLNK Link Network: link graph, forests, trees, ShadowRows, MirrorColumns	Payment processing and escrow (covered in kogi-bank SDD)
CRDT distributed consistency, federation, and conflict resolution (KPCL)	Community messaging and notification delivery (covered in kogi-community SDD)
All 35+ sheet definitions with full column schemas	Third-party provider API implementations (covered in kogi-providers SDD)
View Engine: filters, sorts, groups, pivots, board modes (KPVW)	
Computation Engine: all 20+ analytical models (KPCM)	
kogi-engine integration and plugin writeback protocol	
Persistence, storage, and indexing architecture	

In Scope	Out of Scope
Security, access control, and compliance model	
API surface: REST, gRPC, PQL, WebSocket	
Export, import, and embedding (KPEX)	
Portfolio Template System	
Oba AI integration in the spreadsheet layer	

1.2 Design Philosophy

Every entity in the Kogi ecosystem is a row. Every property is a column. Every perspective is a sheet. Every connection is an edge. Every boundary is a Space. The KIMDSS is not a feature — it is the operating system layer on which all Kogi applications are built.

Principle	Statement	Implementation
Universal Abstraction	Every platform entity — from a single task to a federation — is representable as a PortfolioComponent row	Single PortfolioRow struct; typed ItemType/ContainerType discriminant; universal column schema superset
Spatial Organization	Spaces, Workspaces, and Namespaces provide bounded, governed, discoverable contexts for portfolios and identities	KSPC/KWSP/KNSPC modules; Space as PortfolioContainer; Workspace as active session context; Namespace as hierarchical address
Event-Sourced Truth	Every mutation is an immutable EventLog entry. The current state is the replay of all events. Nothing is ever deleted.	PostgreSQL append-only EventLog; CRDT replay on merge; snapshot + incremental replay for performance
Distributed-First	The system is designed for multi-node, multi-device, offline-capable operation from the ground up	VectorClock per component; LWW + OR-Set CRDT; federation via PortfolioFederation; CrdtLog buffer in Redis
Identity Sovereignty	A Sovereign Entity controls their data, their identities, their visibility, and their connections	KPID multi-identity system; VisibilityMask; SplitPolicy; ProfilePartition; owner-level column encryption
Connected Economy	Every spreadsheet is a node in a global economic graph. Value flows through connections.	KLNK InterPortfolioLink; ShadowRow protocol; MirrorColumn sync; LinkForest traversal API
AI-Augmented Intelligence	The spreadsheet is not passive storage — it actively surfaces intelligence, flags risks, and suggests actions	kogi-engine WritebackService; AIColumn type; Oba AI overlay; anomaly detection pipeline
Open Extensibility	The system is designed to be extended by users, organizations, and third-party developers	PortfolioPlugin trait; FormulaColumn; custom column types; KOGI-APPSTORE template marketplace; kogi-dev SDK

2. Spaces, Workspaces & Namespaces (KSPC · KWSP · KNSPC)

Design Principle: If the PortfolioComponent is the atom and the Master Spreadsheet is the molecule, Spaces are the cells — bounded, governed, contextual environments within which portfolios, identities, and organizations live and collaborate. Workspaces are the active working sessions within a Space. Namespaces are the hierarchical addressing scheme that makes every entity in the ecosystem uniquely and predictably locatable.

2.1 Conceptual Overview

The three spatial abstractions — Space, Workspace, and Namespace — solve three distinct but related problems:

Abstraction	Problem Solved	Analogy	Module
Space (KSPC)	How do we group portfolios, identities, and organizations into bounded, governed, discoverable communities with shared context, shared resources, and shared governance?	A city district — a named area with its own character, members, rules, and shared infrastructure	KSPC
Workspace (KWSP)	How does a user (or agent) establish an active working context — a "desk" — that scopes which sheets, views, and tools are currently in focus, without losing access to the full portfolio?	A physical desk — what's currently open, arranged, and in use	KWSP
Namespace (KNSPC)	How do we assign unique, hierarchical, collision-free addresses to every entity across all users, organizations, spaces, and federation nodes in the platform?	A filesystem path or DNS hierarchy — a globally unambiguous address	KNSPC

2.2 Space (KSPC) — Full Specification

2.2.1 Concept

A Space is a named, governed, bounded digital environment that acts as the top-level social, organizational, and operational context on the Kogi platform. Every Space contains: a member roster with tiered roles, a root portfolio (shared by all members), one or more Workspaces, a set of Rooms (messaging channels), an event feed, a governance structure, and a KLNK link network sub-graph.

Critically, a Space is itself a PortfolioComponent of type Container (ContainerType::Space). This means every Space carries the full ComponentMetadata, CRDT versioning, EventLog, PolicyEngine, and PermissionTier infrastructure of the Portfolio System. A Space is not a separate system — it is a first-class Portfolio entity.

2.2.2 Space Types

Space Type	Description	Default Governance	Portfolio Type	Namespace Prefix
Personal Space	A single user's private portfolio environment — their root Space. Created automatically on account registration. Contains all their portfolios, Workspaces, and private data.	Owner only	Personal Portfolio	@{handle}/
Team Space	A collaborative working environment for a named squad, tribe, guild, or chapter. Members share a project portfolio and Workspace.	Team lead + member consensus	Shared Project Portfolio	team/{name}/
Organization Space	The operational Space for a formally defined organization (LLC, Corp, Trust, Cooperative, Collective, DAO, Federation). Contains all org portfolios, member Workspaces, and governance.	Org governance model (varies)	Organization Portfolio	org/{slug}/
Cooperative Space	A member-owned Space with democratic governance, shared treasury, and revenue distribution mechanisms.	1-member-1-vote	Cooperative Portfolio	coop/{slug}/
Collective Space	An open-contribution Space with light moderation. Any approved member can contribute to the shared portfolio.	Consensus / steward review	Collective Portfolio	collective/{slug}/
Community Space	A public or semi-public topical community — a community of practice, interest group, or professional association.	Admin + community vote	Community Portfolio	community/{slug}/
Federation Space	A meta-Space linking multiple Organizations, Cooperatives, or Collectives for joint governance, shared resources, and cross-entity campaigns.	Federal / representative	Federation Portfolio	fed/{slug}/
Project Space	A Space scoped to a single portfolio project or program — temporary, with a defined lifecycle (start to end) tied to the project timeline.	Project owner	Project Portfolio	project/{id}/
Research Space	A Space for research collaborations: academic, market, or R&D. Supports structured contribution, citation tracking, and dataset management.	PI / research lead	Research Portfolio	research/{slug}/
Event Space	A time-bounded Space for a specific event (conference, hackathon, workshop). Auto-archives after the event lifecycle ends.	Event host	Event Portfolio	event/{id}/
Public Studio	A creator's public-facing Space for showcasing portfolio items, engaging fans/clients, and monetizing content via the Marketplace.	Creator / owner	Creator Portfolio	studio/{handle}/
Private Studio	A personal creative or research environment — private by default. A sandbox for works in progress.	Owner only	Personal Creative Portfolio	@{handle}/studio/

Space Type	Description	Default Governance	Portfolio Type	Namespace Prefix
Platform Space	A system-managed Space for platform-level resources, templates, guides, and announcements. Read-only for non-admin users.	Platform admin	Platform Portfolio	platform/{slug}

2.2.3 Space Data Model

Field	Type	Description
space_id	ComponentId (UUID)	Globally unique identifier. Maps to PortfolioSystem ComponentId. Immutable.
space_type	SpaceType enum	personal team organization cooperative collective community federation project research event studio platform
slug	String (unique)	URL-safe unique handle within namespace. Used in KNSPC address: e.g. team/alpha-squad/
name	String	Display name of the Space
description	RichText	Markdown-formatted description; shown on Space profile
visibility	Visibility	Private Protected Internal Public Unlisted — governs KLNK discovery and search indexing
status	ComponentStatus	Draft Active Paused Archived Dissolved — lifecycle state
namespace_path	NamespacePath	Full hierarchical address: e.g. kogi://org/pamoja-capital/team/engineering/. See §2.4.
members[]	Vec<SpaceMember>	Member roster: { user_id, role, joined_at, contribution_weight, stake_pct, active_workspaces[] }
member_count	ComputedColumn (u32)	Count of active members
roles[]	Vec<SpaceRole>	Custom role definitions beyond the standard PermissionTier set. E.g. "Steward", "Treasurer", "Facilitator".
root_portfolio_id	ComponentId	Root portfolio component for this Space. All Space-scoped portfolio items are children of this component.
workspaces[]	Vec<WorkspaceId>	Active Workspaces within this Space (see §2.3)
rooms[]	Vec<RoomId>	Linked kogi-community Room component IDs. Chat channels, deal rooms, governance rooms.
feed_id	ComponentId	Space-scoped activity feed component
governance_config	GovernanceConfig	Governance model: vote thresholds, quorum rules, proposal types, multi-sig treasury settings

Field	Type	Description
treasury_account_id	Option<AccountId>	kogi-bank account for Space treasury (org/coop/collective types)
link_network_node_id	LinkNodeId	This Space's node in the KLNK graph. All member portfolios link to/from this node.
namespace_registry_id	NamespaceId	The Namespace registry entry for this Space (see §2.4)
toolbox_ids[]	Vec<ToolBoxId>	TMS ToolBoxes attached to this Space (shared tools accessible to all members)
policy_ids[]	Vec<PolicyId>	Governance policies applied to all components within this Space
shared_sheet_ids[]	Vec<SheetId>	Sheets shared across all member Workspaces within this Space
template_id	Option<TemplateId>	Portfolio template applied when this Space was created
federation_links[]	Vec<FederationLinkId>	Cross-Space federation links (for Federation Space type)
metadata	ComponentMetadata	Full ComponentMetadata: id, owners, tags, vector_clock, version, created_at, updated_at, properties

2.2.4 Space Lifecycle

State	Description	Entry Condition	Exit Conditions
Draft	Space is being configured. Not discoverable. No members besides owner.	Space created	Owner publishes → Active
Active	Fully operational. Members can join, contribute, collaborate, and govern.	Owner publishes from Draft	Owner pauses → Paused; dissolution vote → Dissolving
Paused	Temporarily suspended. Members retain access. No new member joins or contributions.	Owner or governance vote	Owner resumes → Active
Restricted	Access limited to existing members. No new joins. Governance events continue.	Compliance flag or governance decision	Restriction lifted → Active
Dissolving	Dissolution process initiated. Asset distribution underway. Final governance actions permitted.	Supermajority dissolution vote (org/coop)	All assets distributed, final report published → Dissolved
Dissolved	Space permanently closed. All assets redistributed or transferred. Archive preserved.	Dissolution process complete	(Terminal)
Archived	Space archived — read-only access, full history preserved. No new activity.	Auto-archive after Event Space end date; manual	Restore → Active (if applicable)

2.2.5 Space Member Roles

Role	PermissionTier	Capabilities
Owner	Owner (6)	Full control: configure Space, manage all members, manage treasury, dissolve Space, manage namespaces, all content
Admin	Admin (7)	Platform-level super-admin override. Accessible only to platform administrators.
Governor	Manager (5)	Submit and vote on governance proposals, manage treasury within approved limits, configure Space policy
Steward	Manager (5)	Review and approve contributions, moderate content, manage member roles below Steward level
Treasurer	Manager (5)	Manage treasury operations: initiate distributions, approve expense claims, manage bank account
Editor	Editor (4)	Create and edit Space portfolio components, create Workspaces, manage Space views and templates
Contributor	Contributor (3)	Submit contributions to shared portfolio, comment on governance proposals, participate in polls and surveys
Member	Contributor (3)	Standard Space membership: access shared Workspaces, view shared sheets, participate in Rooms and events
Viewer	Viewer (1)	Read-only access to Space public/internal content. Cannot contribute or join Rooms.
Guest	Viewer (1)	Temporary read-only access. No contribution rights. Access expires automatically.
Bot	System	Automated agent (Oba, integration bot). Configurable capability set defined per bot registration.

2.3 Workspace (KWSP) — Full Specification

2.3.1 Concept

A Workspace is the active, personalized working context within a Space. Where a Space is persistent and shared, a Workspace is dynamic and personal — it represents what a specific user (or team of users) is currently working on within the Space, with their own arrangement of open sheets, active views, pinned rows, recent components, and tools.

Every user has a Personal Workspace within their Personal Space (auto-created). When they join or create other Spaces, new Workspaces are provisioned for them within those Spaces. A user may have multiple Workspaces open simultaneously (one per Space they are active in) and switch between them using the Workspace switcher.

A Workspace is itself a PortfolioComponent (ContainerType::Workspace) so that its configuration, history, and state are captured in the EventLog and can be restored from snapshots.

2.3.2 Workspace Types

Workspace Type	Description	Scope	Auto-Created?
Personal Workspace	A single user's primary working context within a Space. Contains their open sheets, pinned items, recent components, and tool configuration.	User-scoped within a Space	Yes — on Space join
Shared Workspace	A collaborative Workspace accessible to multiple members simultaneously. Enables screen-sharing-style coordination on the same sheet configuration.	Group-scoped within a Space	On explicit creation
Project Workspace	A Workspace scoped to a specific portfolio project. Contains all project-relevant sheets (Tasks, Timeline, Resources, Budget) pre-configured.	Project-scoped within a Space	On Project item creation (optional)
Guest Workspace	A restricted Workspace for temporary guests in a Space. Read-only access to configured sheets. Auto-expires with guest access.	Guest-scoped within a Space	On guest invitation
Template Workspace	A read-only reference Workspace defining a standard sheet/view arrangement. Used as the starting point for new member Workspaces.	Space-scoped (all members inherit)	On template application
AI Workspace	A Workspace managed by the Oba AI agent for automated monitoring, reporting, and action workflows. Runs scheduled queries and generates briefing reports.	System-scoped within a Space	On Oba workspace activation
Archive Workspace	A read-only Workspace showing archived components within the Space. Provides access to historical data without cluttering the active Workspace.	User-scoped within a Space	On first archive access

2.3.3 Workspace Data Model

Field	Type	Description
workspace_id	ComponentId (UUID)	Globally unique identifier for this Workspace
workspace_type	WorkspaceType enum	personal shared project guest template ai archive
owner_user_id	EntityId	User (or system agent) who owns this Workspace
space_id	ComponentId	Parent Space this Workspace belongs to
name	String	Display name (e.g. "Jordan's Dev Workspace", "Q2 Sprint Board")
open_sheets[]	Vec<SheetRef>	Currently open sheets in tab order: { sheet_id, view_id, scroll_position, active_row_id }
active_sheet_id	Option<SheetId>	Currently focused sheet (the tab in view)
pinned_rows[]	Vec<ComponentId>	Rows pinned to the top of any sheet view in this Workspace (quick access)

Field	Type	Description
pinned_columns[]	Vec<ColumnId>	Columns pinned across all sheet views in this Workspace
recent_components[]	Vec<{ComponentId, ts}>	Recently accessed components (max 50; FIFO eviction)
active_filters[]	Vec<ViewFilter>	Workspace-level filters applied globally across all open sheets
layout_config	LayoutConfig	Workspace layout: { sidebar_visible, toolbar_config, density, theme_override, split_pane_config }
tool_config	WorkspaceToolConfig	Active TMS ToolBoxes and their configuration within this Workspace
klmk_panel_open	bool	Whether the Link Network panel is currently open in this Workspace
active_board_mode	Option<BoardMode>	If a board view is active: Kanban Gantt Calendar Network etc.
session_context	SessionContext	{ device_id, client_version, locale, timezone, last_active_at }
oba_assistant_state	ObaWorkspaceState	Oba's current context in this Workspace: active query, pending actions, briefing state
saved_views[]	Vec<ViewId>	User's saved ViewDefinitions accessible from this Workspace
namespace_path	NamespacePath	KNSPC address of this Workspace: e.g. kogi://org/pamoja/workspace/jordan-dev/
metadata	ComponentMetadata	Full ComponentMetadata with EventLog and CRDT versioning

2.3.4 Workspace Session Protocol

A Workspace session is established when a user authenticates and activates a Workspace. The session protocol coordinates real-time state across devices and collaboration partners:

1. Session Open: Client sends WorkspaceSessionOpen to KWSP service with workspace_id, device_id, client_version, and VectorClock state.
2. State Sync: Server returns current WorkspaceState snapshot + any CrdtLog operations since client's last VectorClock.
3. Active Presence: Client registers as active in the Workspace. Other clients in the same shared Workspace receive PresenceUpdate.
4. Real-Time Stream: Client subscribes to WorkspaceEventStream (WebSocket). All sheet mutations, cursor movements, and Oba actions broadcast to all subscribers.
5. Cursor Sync: User's active row and column broadcast as CursorEvent every 500ms. Other users' cursors rendered as named indicators.
6. Disconnection: On client disconnect, last CursorEvent is retained for 30s then cleared. Workspace session persists with last known state.

7. Session Close: Client sends `WorkspaceSessionClose`. Server archives session metadata to `WorkspaceHistory`.

2.4 Namespace (KNSPC) — Full Specification

2.4.1 Concept

The Namespace system (KNSPC) provides the hierarchical addressing scheme that makes every entity in the Kogi ecosystem uniquely and predictably locatable. A Namespace address (NamespacePath) is a structured, human-readable URI that encodes the entity's position in the organizational hierarchy, its type, and its unique slug within that context.

Namespace addresses follow the URI scheme:

```
kogi://{scope}/{type}/{slug}/{sub-type}/{sub-slug}/...
```

Examples:

<code>kogi://user/jordan-dev/</code>	<code>-- Personal Space</code>
<code>root</code>	
<code>kogi://user/jordan-dev/portfolio/freelance-dev/</code>	<code>-- A specific</code>
<code>portfolio</code>	
<code>kogi://user/jordan-dev/workspace/main/</code>	<code>-- Personal Workspace</code>
<code>kogi://org/pamoja-capital/</code>	<code>-- Org Space root</code>
<code>kogi://org/pamoja-capital/team/engineering/</code>	<code>-- Team Space within</code>
<code>an Org</code>	
<code>kogi://org/pamoja-capital/workspace/q2-sprint/</code>	<code>-- Workspace in Org</code>
<code>kogi://org/pamoja-capital/project/api-redesign/</code>	<code>-- A Project</code>
<code>component</code>	
<code>kogi://coop/writers-guild/sheet/finances/</code>	<code>-- A specific sheet</code>
<code>in a Coop Space</code>	
<code>kogi://fed/shangoOS/org/pamoja/</code>	<code>-- Federated entity</code>
<code>address</code>	
<code>kogi://community/fintech-builders/</code>	<code>-- Community Space</code>
<code>root</code>	
<code>kogi://platform/templates/</code>	<code>-- Platform Space for</code>
<code>templates</code>	

2.4.2 Namespace Registry Schema

The KNSPC Namespace Registry maintains the canonical mapping between NamespacePaths and ComponentIds. It is the platform's DNS-equivalent — the authoritative directory of all named entities.

Field	Type	Description
namespace_id	UUID	Unique identifier for this namespace entry
path	NamespacePath	Full hierarchical path string. Unique across the entire platform.
component_id	ComponentId	The PortfolioComponent (Space, Workspace, Portfolio, etc.) this path resolves to
entity_type	NamespaceEntityType	user org coop collective fed community project team workspace sheet portfolio component platform
slug	String	The terminal segment of the path (e.g. "engineering" in org/pamoja/team/engineering)
parent_namespace_id	Option<UUID>	Parent namespace entry (null for root-level paths)
children_count	ComputedColumn	Count of child namespace entries under this path
visibility	Visibility	Public Protected Internal Private — governs KLNK and search discoverability
aliases[]	Vec<String>	Alternative paths that resolve to the same component (for org renames, redirects)
canonical_url	String	Full canonical URL: https://kogi.io/{entity_type}/{slug}/...
federation_origin	Option<NodeId>	For federated entities: the remote node that owns this namespace entry
created_at	DateTime	Namespace entry creation timestamp
last_resolved_at	DateTime	Last time this path was successfully resolved (for stale detection)

2.4.3 Namespace Resolution Protocol

When a client requests a resource at a NamespacePath, the KNSPC resolver applies the following algorithm:

8. Parse: Split the NamespacePath into segments: [scope, type, slug, sub-type, sub-slug, ...]
9. Root Lookup: Query the NamespaceRegistry for the root-level entry matching scope + type + slug
10. Permission Check: Evaluate the caller's PermissionTier against the root entry's visibility and ACL
11. Sub-path Traversal: If additional path segments exist, traverse the component hierarchy following GraphEdge::Hierarchy edges from the root
12. Final Resolution: Return the ComponentId of the terminal path segment's entry
13. Cache: Cache the resolution result in Redis with a TTL of 300s (5 min) for hot-path performance
14. Redirect Handling: If the path matches an alias[], return a 301-equivalent redirect to the canonical path
15. Federation: If the root scope indicates a federation origin (kogi://fed/...), forward resolution to the appropriate remote node via PortfolioFederation

2.4.4 Namespace Operations

Operation	Description	Permission Required	Namespace Effect
CreateNamespace	Register a new NamespacePath → ComponentId mapping	Owner of parent namespace	Adds entry to NamespaceRegistry; notifies KLNK graph
RenameNamespace	Change the slug of an existing namespace entry	Owner of the namespace	Creates an alias for the old path; canonical path updated; all references updated via async migration
DeleteNamespace	Remove a namespace entry (only if component is archived)	Owner of the namespace	Entry removed; aliases tombstoned; KLNK edges updated
AliasNamespace	Add an alternative path that resolves to the same component	Owner of the namespace	Alias added; canonical path unchanged
FederateNamespace	Publish a namespace entry to a federation peer node	Federation admin	Entry replicated to peer node's NamespaceRegistry
ClaimNamespace	Claim ownership of a previously unclaimed slug	First valid claimer	Entry created; ownership assigned
TransferNamespace	Transfer ownership of a namespace entry to another entity	Current owner	Ownership updated; history preserved; recipient notified

2.5 Space-Workspace-Namespace Integration

Spaces, Workspaces, and Namespaces are not independent systems — they form a layered architecture where each level depends on the layers below it:

Layer	Component	Namespace Address	Contains	Governed
L0 — Identity Root	Sovereign Entity	kogi://user/{handle}/ or kogi://org/{slug}/	All Spaces for this entity	Entity own
L1 — Space	Space (KSPC)	kogi://{scope}/{slug}/	Workspaces, Portfolios, Rooms, Feed, Governance	Space gov config; Pol
L2 — Workspace	Workspace (KWSP)	kogi://{scope}/{slug}/workspace/{ws-slug}/	Open Sheets, Pinned Rows, Tools, Session state	Workspac session pr
L3 — Portfolio	Portfolio (KPMS)	kogi://{scope}/{slug}/portfolio/{p-slug}/	Programs, Projects, Assets, all PortfolioComponents	Portfolio P Permission
L4 — Sheet	Sheet / View (KPVW)	kogi://{scope}/{slug}/sheet/{sheet-id}/	PortfolioRows filtered/sorted/grouped by definition	Sheet visi Permission

Layer	Component	Namespace Address	Contains	Governed
L5 — Component	PortfolioComponent	kogi://{scope}/{slug}/component/{comp-slug}/	Cells, EventLog, CrdtLog, GraphEdges	Component, PolicyEngine, VisibilityManager

2.6 Space as Portfolio Container

Every Space is simultaneously a governance unit AND a Portfolio Container. When a Space is created, the Portfolio System automatically provisions:

- A root Portfolio component (item_type: Portfolio) whose namespace_path matches the Space's root path
- A default set of sheets determined by the Space type (e.g. Organization Space gets: Projects, Finances, Members, Governance, Shared Portfolios, Link Network)
- A default Template Workspace for the Space owner, pre-loaded with those sheets
- A KLNK link node for the Space, allowing other users to link to/from the Space
- A Namespace entry in the NamespaceRegistry mapping the Space's slug to its ComponentId
- A GovernanceConfig based on the Space type (configurable post-creation)
- A Treasury Account in kogi-bank if the Space type includes economic activity (org, coop, collective, federation)

2.7 Cross-Space Portfolio Connections

The combination of Spaces and the KLNK Link Network enables cross-Space portfolio connections — the mechanism by which portfolios in different Spaces become interconnected:

Connection Type	Description	How Established	Spreadsheet Effect
Space Membership	A user joins a Space. Their Personal Space's root portfolio gains an OrgMembership KLNK edge to the Space's root portfolio.	User accepts Space invitation	Space's shared components appear in user's SHT-020 (Shared Portfolios)
Cross-Space Project	A Project component in Space A has collaborators from Space B. The project's namespace is accessible from both Spaces.	Collaborators invitation + acceptance	ShadowRow of the project created in Space B members' Workspaces
Space Federation	Two or more Spaces federate: their portfolios are CRDT-synced and their KLNK graphs are merged.	Governance vote in both Spaces + Federation setup	All federated components appear cross-Space; VectorClock merged
Space Marketplace Link	A component from Space A is listed on the Marketplace. Buyers from any Space can link to it.	Listing published with visibility=Public	InvestedIn or Contracted KLNK edge created;

Connection Type	Description	How Established	Spreadsheet Effect
			ShadowRow in buyer's Workspace
Shared Resource Space	A Resource Space (e.g. a compute pool or license registry) is linked to multiple Spaces for shared access.	ResourceShares KLNK edge from resource to Spaces	Resource rows appear as ShadowRows in all linked Spaces' Resource sheets
Cross-Space Campaign	A fundraising campaign spans multiple Spaces (e.g. a federation-wide capital raise).	Campaign created in Federation Space; Space members join	Campaign row appears in all member Spaces' Equity & Crowdfunding sheets

3. System Architecture

3.1 Eight-Layer Architecture

Layer	Name	Module	Components
L0	Substrate	KPSS (Rust)	PortfolioComponent · ComponentMetadata · ComponentData · EventLog · CrdtLog · VectorClock · GraphEdge · PolicyEngine · ToolBoxId
L1	Spatial Layer	KSPC/KWSP/KNSPC (Go)	Space · Workspace · NamespaceRegistry · SpaceMember · GovernanceConfig · Treasury · FederationLink
L2	Registry	KPRG (Go + PG)	ComponentIndex · OwnerIndex · TagIndex · TypeIndex · StatusIndex · NamespaceIndex · LinkNetworkIndex · FullTextIndex
L3	Spreadsheet Engine	KPSS (Rust + Go)	PortfolioRow · ColumnSchema · SheetDefinition · CellStore · ColumnComputer · AggregationEngine · ViewFilter/Sort/Group
L4	View Engine	KPVW (Go)	ViewDefinition · BoardMode · PivotConfig · ChartConfig · RowHighlightRule · PinnedColumns · SavedViews · SpaceViewOverlay
L5	Intelligence	kogi-engine (Scala)	AIColumn · WritebackService · HealthScore · RiskScore · MatchScore · RecommendationEngine · AnomalyEngine · ObaOverlay
L6	Link Network	KLNK (Go + Rust)	LinkNode · LinkEdge · LinkForest · LinkTree · ShadowRow · MirrorColumn · VisibilityProtocol · SpaceLinkSubgraph
L7	Identity	KPID (Go)	SovereignEntity · Account · ProfilePartition · VisibilityMask · SplitPolicy · CrossIdentityMerge · IdentityAnchor

3.2 Service Architecture

Service	Language	Responsibilities	Ports / Protocols
portfolio-service	Go	Core PortfolioComponent CRUD, GraphEdge management, PolicyEngine evaluation, ResourceAllocation, ActionKind dispatch	REST :8080 · gRPC :9200
spreadsheet-service	Go	Sheet rendering, ViewDefinition management, row query (PQL), aggregation, export coordination	REST :8081 · gRPC :9201 · WS :8091
space-service	Go	Space CRUD, member management, governance actions, Space lifecycle, treasury provisioning, namespace registration	REST :8082 · gRPC :9202

Service	Language	Responsibilities	Ports / Protocols
workspace-service	Go	Workspace CRUD, session management, sheet tab state, Workspace event streaming, presence protocol	REST :8083 · gRPC :9203 · WS :8093
namespace-service	Go	KNSPC registry, path resolution, alias management, federation namespace sync	REST :8084 · gRPC :9204
link-network-service	Go	KLNK LinkEdge management, ShadowRow provisioning/sync, MirrorColumn management, link graph traversal API	REST :8085 · gRPC :9205
identity-service	Go	KPID ProfilePartition management, VisibilityMask, account-partition mapping, CrossIdentityMerge, SplitPolicy enforcement	REST :8086 · gRPC :9206
crdt-service	Rust	CrdtLog management, operation application, merge coordination, federation sync, conflict detection and surfacing	gRPC :9207 · Kafka consumer/producer
computation-service	Rust+Scala	ComputedColumn evaluation (sync tier), FormulaColumn ColumnComputer, aggregation engine, rollup computation	gRPC :9208 (internal)
writeback-service	Go	Engine writeback protocol: receive AI-computed values from kogi-engine, validate permissions, write to CellStore, broadcast updates	gRPC :9209 (internal kogi-engine only)
export-service	Go	KPEX export pipeline: CSV, XLSX, JSON, JSON-LD, PDF, vCard, GDPR archive, webhook dispatch, embed configuration	REST :8087
snapshot-service	Go	Portfolio snapshot creation, checkpoint management, restore orchestration, S3 coordination	REST :8088 · gRPC :9210
search-service	Go	Full-text and faceted search across all PortfolioComponents; Meilisearch/Typesense coordination; personalized re-ranking via kogi-engine	REST :8089
kogi-engine	Scala 3	All intelligence sub-engines: Analytics, Telemetry, Recommendation, Personalization, Match, Graph, Risk, Optimization, Benefits, Grant, KLNK	gRPC :9100 · Kafka consumer/producer
api-gateway	Go	Authentication, authorization, rate limiting, request routing, CORS, API versioning, developer API key management	REST :443 · WS :443

3.3 Data Flow Architecture

All data flows between services are either synchronous (gRPC for low-latency reads/writes) or asynchronous (Kafka for event streaming, analytics, and cross-service propagation). The following diagram describes the primary data flow paths:



4. Core Data Model

4.1 PortfolioComponent (Rust)

The PortfolioComponent is the universal node of the entire KIMDSS. Every row in every sheet, every Space, every Workspace, every sheet definition, every link node — is a PortfolioComponent. The Rust struct definition is the authoritative schema:

```
// portfolio_system.rs — Core Component struct
pub struct Component {
    pub metadata: ComponentMetadata,
    pub data: ComponentData,
    pub item_book: Option<ItemBookData>,
}

pub struct ComponentMetadata {
    pub id: ComponentId, // UUID — immutable
    pub owners: Vec<UserId>,
    pub tags: HashSet<String>,
    pub policy_ids: Vec<PolicyId>,
    pub created_at: DateTime<Utc>,
    pub updated_at: DateTime<Utc>,
    pub last_actor: String, // node_id:identity_tag for CRDT tiebreak
    pub vector_clock: VectorClock,
    pub properties: HashMap<String, serde_json::Value>,
    pub version: VersionString, // semver
    pub budget: f64,
    pub budget_spent: f64,
    pub resource_units: f64,
    pub namespace_path: Option<NamespacePath>, // KNSPC address
    pub space_id: Option<ComponentId>, // parent Space (KSPC)
    pub workspace_ids: Vec<ComponentId>, // open Workspaces (KWSP)
    pub identity_tags: Vec<IdentityTag>, // KPID partition tags
}

pub struct ComponentData {
    pub category: ComponentCategory, // Item | Container
    pub name: String,
    pub description: String,
    pub status: ComponentStatus,
    pub state: ComponentState,
    pub visibility: Visibility,
    pub children: Vec<ComponentId>,
    pub parents: Vec<ComponentId>,
}
```

```

pub links:          Vec<ComponentId>,
pub dependents:     Vec<ComponentId>,
pub dependencies:   Vec<ComponentId>,
pub users:          ComponentUsers,
pub analytics:      ComponentAnalytics,
pub risks:          Vec<Risk>,
pub hashtags:       HashSet<String>,
pub topics:         HashSet<String>,
pub toolbox_ids:    Vec<ToolBoxId>,
pub plugin_configs: HashMap<String, serde_json::Value>,
pub space_context:  Option<SpaceContext>, // KSPC integration
pub link_metadata:  Option<LinkMetadata>, // KLNK integration
}

```

4.2 Space Context Extension

Components that belong to a Space carry a SpaceContext struct that records their Space membership, the Workspaces they are currently open in, and their governance status within the Space:

```

pub struct SpaceContext {
    pub space_id:          ComponentId,
    pub space_type:        SpaceType,
    pub namespace_within:  NamespacePath,
    pub governance_status: SpaceGovernanceStatus,
    // GovernanceStatus: Active | PendingApproval | UnderReview | Suspended
    pub contribution_weight: f64, // for shared portfolio attribution
    pub open_workspaces:    Vec<WorkspaceRef>,
    // WorkspaceRef { workspace_id, user_id, active: bool }
}

pub struct LinkMetadata {
    pub link_node_id:      LinkNodeId,
    pub inbound_link_count: u32,
    pub outbound_link_count: u32,
    pub shadow_row_ids:    Vec<ShadowRowId>, // ShadowRows in OTHER users'
    // spreadsheets
    pub has_shadow_of:     Option<ShadowRowId>, // If THIS row IS a ShadowRow
    pub link_visibility:   LinkVisibility,
    // LinkVisibility: Public | Followers | Connections | SpaceMembers | Private
}

```

4.3 ComponentMetadata — Extended Field Reference

Field	Type	Description	CRDT Semantics
id	UUID (ComponentId)	Globally unique immutable identifier. Never changes after creation. Used as PK in all stores.	Identity — never mutated by CRDT
owners	Vec<UserId>	One or more users/orgs with Owner-level permission. Multi-owner enables shared ownership for cooperative assets.	OR-Set: concurrent adds survive; removes require explicit tag
tags	HashSet<String>	Flat string tag set for discovery, filtering, and classification.	OR-Set: concurrent adds survive
policy_ids	Vec<PolicyId>	Active governance policies. Evaluated by PolicyEngine on every mutation.	OR-Set: concurrent policy attach/detach both survive
created_at	DateTime<Utc>	Immutable creation timestamp. Set by Substrate on create_item() / create_container().	LWW — but practically immutable (set once, never mutated)
updated_at	DateTime<Utc>	Last mutation timestamp. Updated atomically with every cell write.	LWW: highest-timestamp write wins
last_actor	String	Node ID + identity tag of the last writer. Used as tiebreaker when VectorClock timestamps are equal.	LWW: highest-timestamp write wins; actor is metadata
vector_clock	VectorClock	HashMap<NodeId, u64>. Logical clock for causal ordering. Incremented on every mutation at the writing node.	Merge: element-wise max of all node counters
properties	HashMap<String, JSON>	Extensible key-value metadata. Custom columns are stored here under their column_key.	LWW per key: each key independently LWW-settled
version	VersionString	Semver string managed by bump_version(). Manual or auto-incremented on significant mutations.	LWW: highest-timestamp write wins
budget	f64	Total budget allocated via allocate_budget(). Updated by ResourceAllocation.	LWW: updated by allocation events
budget_spent	f64	Cumulative spend recorded via record_spend(). Monotonically increasing.	LWW: only increases; conflict = higher value wins
resource_units	f64	Generic resource unit counter for non-monetary resources.	LWW: higher value wins
namespace_path	Option<Namespace Path>	KNSPC address. Set on Space/Workspace/Portfolio creation; updated on rename.	LWW: alias management handled by namespace-service

Field	Type	Description	CRDT Semantics
space_id	Option<ComponentId>	Parent Space this component belongs to. Null for Personal Space root.	LWW: set on Space membership; updated on transfer
workspace_ids	Vec<ComponentId>	Workspaces currently displaying this component as an open tab.	OR-Set: Workspace open/close operations add/remove
identity_tags	Vec<IdentityTag>	KPID identity partition tags. Determines which ProfilePartition(s) this row belongs to.	OR-Set: tag add/remove; cross-partition write policy enforced

4.4 ComponentData — Core Fields

Field	Type	Values / Description	CRDT Semantics
category	ComponentCategory	Item(ItemCategory) Container(ContainerCategory)	LWW — set at creation; rarely changes
name	String	Primary display name. Shown everywhere.	LWW: highest-timestamp write wins
description	String (Markdown)	Rich-text description. Markdown-compatible.	LWW: highest-timestamp write wins
status	ComponentStatus	Draft Active Paused Completed Archived Cancelled Suspended UnderReview Rejected Custom	Custom lattice (lifecycle order, not LWW) — see §4.5
state	ComponentState	Initializing Configured Running Idle Blocked Failing Recovering Migrating Locked Sealed Custom	LWW within valid state transitions
visibility	Visibility	Private Protected Internal Public Unlisted DraftOnly	LWW: highest-timestamp write wins; PolicyEngine validates
children	Vec<ComponentId>	Direct children in GraphEdge::Hierarchy	OR-Set: attach/detach operations
parents	Vec<ComponentId>	Parent components (may have multiple for shared/collaborative components)	OR-Set: attach/detach operations
links	Vec<ComponentId>	Soft lateral associations (GraphEdge::Link)	OR-Set

Field	Type	Values / Description	CRDT Semantics
dependents	Vec<ComponentId>	Components that depend on this one (inverse of dependencies)	OR-Set: cycle detection on add
dependencies	Vec<ComponentId>	Components this one depends on. add_dependency() checks for cycles.	OR-Set: cycle detection on add
users	ComponentUsers	Tiered user registry: owners, editors, contributors, viewers, watchers, subscribers, followers, investors, donors	Per-set OR-Set: each user set independently managed
analytics	ComponentAnalytics	Engagement counters: views, likes, comments, shares, followers, saves, ctr, engagement_rate, spread	Counter (CRDT counter: always-increment, never decrement directly)
risks	Vec<Risk>	Risk records with severity, probability, mitigation owner, status	OR-Set: risks added/resolved/removed independently
hashtags	HashSet<String>	Social discovery hashtags	OR-Set
topics	HashSet<String>	Semantic topic classifications	OR-Set
toolbox_ids	Vec<ToolBoxId>	TMS ToolBox references (v2.1)	OR-Set: attach/detach
space_context	Option<SpaceContext>	Space membership context (§4.2)	LWW per field within SpaceContext struct
link_metadata	Option<LinkMetadata>	KLNK integration metadata (§4.2)	Counter for counts; LWW for link_visibility

4.5 Status CRDT Lattice

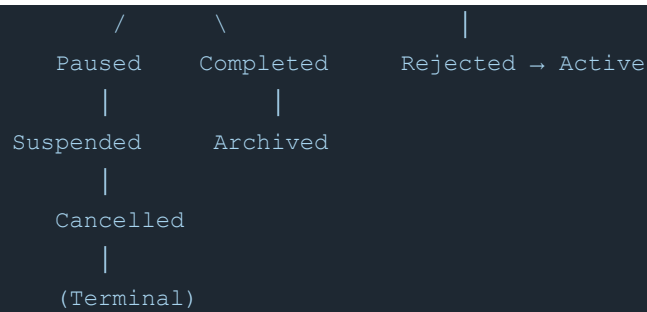
ComponentStatus cannot use simple Last-Write-Wins semantics because status transitions carry business logic, governance requirements, and side effects. The CRDT status lattice defines a partial order over status values such that merge resolves to the "most advanced" status in cases of concurrent transitions:

```
// Status partial order (lattice for CRDT merge)
// A status dominates B if A is "further along" the lifecycle
// Concurrent transitions to sibling statuses resolve to the JOIN (least upper bound)
```

```

Draft
 |
Active ————— UnderReview

```



```

// Merge rule:
// If concurrent: join(A, B) = least upper bound in the lattice
// If A = Active, B = Paused (concurrent): join = Active (Active dominates Paused)
// If A = Active, B = Completed (concurrent): join = Completed (further along)
// If both are Terminal (Archived, Cancelled): join = the one with higher timestamp
// If A = Active, B = UnderReview: join = UnderReview (governance takes precedence)

// PolicyEngine always evaluated BEFORE status transition
// Invalid transitions (e.g. Completed → Active without explicit restore) rejected
  
```

5. Link Network System (KLNK)

The KLNK Link Network is the global inter-portfolio graph of the Kogi platform. Every relationship between every portfolio, identity, Space, and federation node is an edge in this graph. The full graph — traversed as forests and trees — is the complete, queryable picture of how all independent work on the platform is interconnected.

5.1 Graph Model

Concept	Description	Stored In
LinkNode	Any entity that can be a vertex in the KLNK graph. Types: PortfolioComponent Profile Identity Organization Space Workspace Portfolio FederationNode	PostgreSQL link_nodes table
LinkEdge	A typed, directional, weighted edge between two LinkNodes. The atomic unit of inter-portfolio connection.	PostgreSQL link_edges table + Redis hot path
LinkForest	The complete set of all LinkTrees rooted at a given Sovereign Entity. The "full network view" from one entity's perspective.	Computed on demand; hourly materialized view
LinkTree	A rooted, directed subgraph of the KLNK graph from a single root node to a configured max_depth, optionally filtered by edge type.	Computed on traversal query
SpaceLinkSubgraph	The sub-graph of the KLNK graph scoped to a single Space: all edges between Space members and Space-owned components.	Materialized view per Space; updated on membership changes
ShadowRow	A read-only row in User B's spreadsheet that mirrors a linked component from User A's spreadsheet. Created by consent-based cross-portfolio LinkEdges.	PostgreSQL shadow_rows table
MirrorColumn	A specific column from a ShadowRow's source component, synced into the host user's spreadsheet as a ComputedColumn (read-only).	PostgreSQL mirror_columns table; values in CellStore
LinkSubscription	A registered listener that receives webhook or Kafka events when a linked component or edge changes.	PostgreSQL link_subscriptions table

5.2 LinkEdge Data Model

```
pub struct LinkEdge {
    pub edge_id:      EdgeId,           // UUID
    pub from_node:    LinkNodeId,       // source node
    pub to_node:      LinkNodeId,       // target node
    pub edge_type:    LinkEdgeType,     // typed relationship
    pub direction:    EdgeDirection,    // Outbound | Inbound | Mutual
}
```

```

    pub weight:          f64,                // 0.0-1.0, used in graph algorithms
    pub visibility:      LinkVisibility,     // Public | Followers | Connections |
SpaceMembers | Private
    pub consent_status:  ConsentStatus,     // Pending | Accepted | Declined |
Revoked
    pub space_scope:     Option<ComponentId>, // if edge is scoped to a Space
    pub shadow_row_id:   Option<ShadowRowId>, // ShadowRow created by this edge
    pub metadata:        HashMap<String, Value>, // edge-type-specific data
    pub created_at:      DateTime<Utc>,
    pub updated_at:      DateTime<Utc>,
    pub expires_at:      Option<DateTime<Utc>>, // for time-limited edges (Guest,
Temp)
}

pub enum LinkEdgeType {
    Follows,           // A follows B's profile/portfolio
    Subscribes,       // Paid/privileged follow
    Watches,          // A watches a specific component of B
    Collaborates,     // Bidirectional co-contribution
    InvestedIn,       // A invested capital in B's campaign/instrument
    Donated,          // A donated to B's cause
    Contracted,       // A and B have an active or historical contract
    DependsOn,        // A's component depends on B's component
    ResourceShares,   // A shares a resource with B
    FederationPeer,   // Two PortfolioSystem nodes federated for CRDT sync
    OrgMembership,    // User is member of org Space
    SpaceMembership,  // User is member of a Space
    Mentors,          // A mentors B
    Endorses,         // A endorses a skill or component of B
    References,       // A cites/references B's component
    CrossSpaceProject, // A project spans two or more Spaces
    SharedResource,   // A resource Space shares with multiple Spaces
    CampusLink,       // Platform-initiated link (onboarding, templates)
    WorkspaceShares,  // A Workspace is shared with another user
    NamespaceAlias,   // A namespace path aliases to another
    Custom(String),   // Developer-defined edge type
}

```

5.3 ShadowRow Protocol — Full Specification

Step	Description	Service	Data Effect
1. Link Request	User A creates a LinkEdge to User B's component. Edge type determines if ShadowRow consent is required.	link-network-service	LinkEdge created with consent_status=Pending (if consent required) or

Step	Description	Service	Data Effect
			Accepted (if auto-consent type)
2. Consent Invitation	User B receives KNTF notification: "User A wants to link to your component. Configure what they can see."	link-network-service + kogi-community	Notification dispatched; ConsentRequest record created
3. Visibility Config	User B configures: visible_columns (which columns A can see in their ShadowRow) and write_back_columns (which columns A can write back to).	link-network-service (UI: workspace-service)	ConsentConfig stored against the pending LinkEdge
4. Consent Granted	User B accepts. LinkEdge.consent_status → Accepted.	link-network-service	ShadowRow provisioned in A's spreadsheet in the appropriate sheet
5. ShadowRow Provision	ShadowRow created in A's PortfolioComponent registry. status=Active. source_component_id = B's component.	portfolio-service	ShadowRow row inserted in A's component table; indexed as shadow type
6. Initial Sync	MirrorColumn values populated from B's current component state for the agreed visible_columns.	link-network-service → writeback-service	CellStore populated for each MirrorColumn in A's ShadowRow
7. Change Propagation	When B's component changes: EventLog event published to Kafka topic link.shadow-sync. A's ShadowRow receives delta update for visible_columns.	crdt-service → Kafka → link-network-service	MirrorColumn values updated in A's CellStore; WebSocket broadcast to A's Workspace
8. Write-back	If A has write_back_columns permission: A's edits to those columns are propagated back to B's component as a cross-portfolio CrdtOperation.	link-network-service → crdt-service	CrdtOperation tagged with A's identity applied to B's component EventLog; appears in B's sheet
9. Disconnection	LinkEdge deleted. ShadowRow.status → Disconnected. Last known values preserved.	link-network-service → portfolio-service	ShadowRow marked Disconnected; A notified; historical data preserved in EventLog
10. Archive/Delete	A chooses to Archive (preserve) or Delete (remove) the disconnected ShadowRow.	portfolio-service	ShadowRow archived or soft-deleted; EventLog entry appended

5.4 Link Forest & Tree Traversal

Traversal Mode	Algorithm	Max Depth	Use Case
Ego Network	BFS from root node; collect all nodes at depth ≤ 2 with all edges between them	2	Social graph overview; mutual connections
Link Forest	BFS from all root nodes owned by entity; union of all trees	3	Full connectivity overview

Traversal Mode	Algorithm	Max Depth	Use Case
Component Tree	DFS from a single component; all inbound + outbound edges	5	Component-level network context
Dependency Tree	Topological sort on DependsOn edges; cycle-safe (detect_cycle() before traversal)	unlimited	Cross-portfolio critical path analysis
Investment Graph	BFS filtering on InvestedIn + Contracted edges only	3	Financial network visualization
Space Subgraph	All nodes within a Space's SpaceLinkSubgraph; all edges between them	4	Space community structure
Federation Graph	All FederationPeer edges across all PortfolioSystem nodes	full	Platform topology; CRDT sync health
Collaboration Web	Bipartite graph: users ↔ shared components via Collaborates edges	3	Team composition; contribution attribution
Shortest Path	Dijkstra on weighted graph; edge weight = LinkEdge.weight	computed	Degrees of separation; introduction path
Community Detection	Louvain algorithm on undirected projection of full KLNK graph	full graph	Ecosystem cluster analysis
Centrality Computation	PageRank / betweenness centrality on full KLNK graph (sampled for scale)	full graph	Influence scoring; network health

5.5 Space Link Subgraph

Each Space maintains a materialized sub-graph (SpaceLinkSubgraph) of all KLNK edges between nodes within that Space. This enables efficient in-Space graph queries without traversing the full global KLNK graph:

```
pub struct SpaceLinkSubgraph {
    pub space_id:      ComponentId,
    pub nodes:         Vec<LinkNode>,      // all link nodes owned by Space
members
    pub edges:         Vec<LinkEdge>,      // all edges between those nodes
    pub external_edges: Vec<LinkEdge>,      // edges connecting to nodes outside
the Space
    pub last_materialized: DateTime<Utc>,
    pub member_count:  u32,
    pub total_node_count: u64,
    pub total_edge_count: u64,
    pub edge_type_distribution: HashMap<LinkEdgeType, u64>,
}
```

```
// Materialized hourly; updated on:  
//   - Space member join/leave  
//   - LinkEdge create/delete involving Space members  
//   - SpaceMembership edge create/delete
```

6. Multi-Identity System (KPID)

6.1 Identity Hierarchy

Level	Concept	Description	Spreadsheet Mapping
L0	Sovereign Entity	The real-world person, org, or collective that owns one root PortfolioSystem instance	One root PortfolioSystem = one Master Spreadsheet
L1	Account	A login credential set. Multiple accounts per Sovereign Entity. Each has a configurable access scope.	Accounts authenticate to the same root spreadsheet; VisibilityMask scopes their view
L2	Identity	A named, purpose-specific @handle with its own display name, avatar, and bio.	Each Identity is a ProfilePartition + a KLNK LinkNode
L3	ProfilePartition	A logical partition of the root spreadsheet's rows tagged to one Identity. Not a physical copy.	OR-Set of identity_tags on each PortfolioRow determines partition membership
L4	VisibilityMask	Per-account or per-profile rules specifying which rows/columns are visible to which observer type.	Applied at query time on every GET via PostgreSQL RLS + application filter
L5	SplitPolicy	Governance policy on cross-partition data access. Enforced by PolicyEngine at write time.	PolicyEngine.evaluate() called on every cross-partition CrdtOperation

6.2 ProfilePartition Data Model

```
pub struct ProfilePartition {
    pub partition_id:          UUID,
    pub identity_handle:       String,          // @handle - unique within Sovereign
Entity
    pub sovereign_entity_id:   EntityId,
    pub display_name:          String,
    pub avatar_ref:            Option<FileId>,
    pub bio:                   Option<String>,
    pub visibility_mask:       VisibilityMask,
    pub identity_tag:          IdentityTag,     // tag applied to rows in this
partition
    pub isolation_level:       IsolationLevel,
    // IsolationLevel: None | SoftIsolated | HardIsolated
    //   None: cross-identity reads permitted for authenticated owner
    //   SoftIsolated: cross-identity reads logged as CrossPartitionAccessEvent
    //   HardIsolated: cross-identity reads blocked at Substrate layer
    pub linked_accounts:       Vec<AccountId>,
}
```

```

    pub linked_spaces:      Vec<ComponentId>, // Spaces this identity is
member of
    pub public_sheet_view_id: ViewId,
    pub follower_sheet_view_id: ViewId,
    pub connection_view_id:  ViewId,
    pub link_network_node_id: LinkNodeId,    // KLNK node for this identity
    pub namespace_path:      NamespacePath, // kogi://user/{handle}/
    pub split_policy_id:      Option<PolicyId>,
    pub created_at:           DateTime<Utc>,
    pub updated_at:           DateTime<Utc>,
}

pub struct VisibilityMask {
    pub public_rows:      RowFilter,    // filter determining public rows
    pub follower_rows:    RowFilter,    // filter for follower-visible rows
    pub connection_rows:  RowFilter,    // filter for trusted connections
    pub public_columns:   Vec<ColumnId>, // columns visible to public
    pub follower_columns: Vec<ColumnId>,
    pub connection_columns: Vec<ColumnId>,
    pub owner_only_columns: Vec<ColumnId>, // always hidden from non-owners
    pub custom_rules:      Vec<VisibilityRule>, // (condition → column_set)
}

```

6.3 CrossIdentityMerge Protocol

Step	Description	Data Effect	Reversible?
1. Pre-flight Check	Validate that both source and target partitions belong to the same Sovereign Entity. Check for tag conflicts.	No data change	N/A
2. Checkpoint	Create a pre-merge snapshot of both partitions' row sets. Stored in S3 with merge_checkpoint tag.	Snapshot written to S3	Yes — full restore from checkpoint
3. Tag Re-assignment	For all rows tagged only to the source partition (not shared with other partitions): re-tag from source_tag to target_tag.	identity_tags OR-Set operations on rows	Partial — checkpoint restore
4. Shared Tag Handling	For rows tagged to both source and target: source_tag removed; they remain in target.	identity_tags OR-Set remove for source_tag	Partial
5. Namespace Merge	Create a namespace alias from source @handle to target @handle. Source @handle redirects to target.	NamespaceRegistry alias created	Yes — alias can be removed

Step	Description	Data Effect	Reversible?
6. KLNK Node Merge	Merge source identity's KLNK LinkNode into target identity's node. All source's edges re-pointed to target node.	LinkEdge updates; source LinkNode archived	No — edge re-pointing is permanent
7. Space Membership	Update Space memberships: source identity memberships transferred to target.	SpaceMember records updated	Yes
8. Account Update	Re-associate accounts linked to source partition to target partition.	ProfilePartition.linked_accounts updated	Yes
9. Archive Source	Archive source ProfilePartition. It is not deleted — full history preserved.	ProfilePartition.status = Archived	Yes — partition can be restored
10. Audit Entry	Append IdentityMergeEvent to Sovereign Entity's master EventLog.	EventLog append — immutable	N/A (audit only)

7. Sheet System — All 38 Sheets

A Sheet is a named, scoped, materialized view of the PortfolioComponent registry. Sheets are not separate databases — they are defined views (SheetDefinition) evaluated against the single underlying PortfolioRow store. Each sheet is identified by a code (SHT-NNN), has a default column schema, default sort/group, and may have sheet-specific ComputedColumns not available on other sheets.

7.1 Complete Sheet Registry

Sheet ID	Name	Primary Row Types	Default Group	Space-Scoped?
SHT-001	Master Registry	All types	Type → Status	Optional (can be global or Space-filtered)
SHT-002	Portfolio Hierarchy	Portfolio, SubPortfolio, Program, Project	Hierarchy tree (tree)	Optional
SHT-003	Programs	Program	Status	Yes — shows programs in active Space
SHT-004	Projects	Project	Program → Status	Yes
SHT-005	Tasks & Backlog	Task, Story, Epic, Feature	Project → Sprint	Yes
SHT-006	Resources	Resource	Resource Type	Yes
SHT-007	Assets	Asset	Asset Type	Optional
SHT-008	Artifacts	Artifact	Project → Artifact Type	Yes
SHT-009	Finances	All financial rows	Quarter → Category	Optional (always owner-level scoped)
SHT-010	Budget Tracker	Program, Project, Gig, Contract	Program → Status	Yes
SHT-011	Work & Gigs	Gig, Contract, Job, Task	Platform → Status	Optional
SHT-012	Deliverables	Project, Artifact, Release	Project → Status	Yes
SHT-013	Timeline	All dated rows	Quarter → Program	Optional
SHT-014	Roadmap	Program, Project, Milestone	Program → Quarter	Yes
SHT-015	Portable Benefits	BenefitAccount	Benefit Type	No (always Personal Space scoped)
SHT-016	Grants & Microfinancing	Grant, Campaign (grant type)	Status	Optional

Sheet ID	Name	Primary Row Types	Default Group	Space-Scoped?
SHT-017	Equity & Crowdfunding	Campaign, Investment	Campaign Type	Optional
SHT-018	Group Economics	Shared Portfolio, Revenue Pool, Org	Organization	Yes (Org/Coop Space)
SHT-019	Organizations	Profile (org type)	Org Type	Optional
SHT-020	Shared Portfolios	Portfolio (shared)	Owner Org	Yes
SHT-021	Collaboration	All shared/contributed rows	Contributor	Yes
SHT-022	Risk Register	All rows with risk_flags	Risk Severity	Yes
SHT-023	Analytics Dashboard	All rows	Type → Health Score	Optional
SHT-024	Feed & Community	Profile, Post, Artifact (public)	Space	Yes
SHT-025	Contacts & Network	Profile, Contact	Relationship Type	Optional
SHT-026	Marketplace Listings	Asset, Resource, Gig (public)	Category	Optional
SHT-027	Exchange	Investment, Deal, Campaign, Asset	Exchange Type	Optional
SHT-028	Archive	All archived rows	Archive Date	Optional
SHT-029	Event Log	EventLog entries (read-only)	Date → Actor	Optional (filtered by component ownership)
SHT-030	Snapshots	Snapshot records	Date	No (always Personal Space scoped)
SHT-031	Link Network	LinkEdge records	Edge Type → Target	Optional
SHT-032	Identity & Profiles	ProfilePartition records	Identity	No (always Personal Space scoped)
SHT-033	Portable Benefits Deep Dive	BenefitAccount + Gig (contribution)	Benefit Type → Platform	No (Personal Space)
SHT-034	Custom Sheet	User-defined	User-defined	Optional
SHT-035	Template Library	ItemBook:Template rows	Category	Yes (Space templates visible to members)
SHT-036	Spaces & Workspaces	Space (KSPC), Workspace (KWSP) rows	Space Type	No (global view of all the entity's Spaces)
SHT-037	Namespace Directory	NamespaceRegistry entries	Entity Type → Scope	Optional
SHT-038	Workspace History	WorkspaceSession records (read-only)	Date → Workspace	Yes (per-Workspace history)

7.2 SHT-036: Spaces & Workspaces Sheet

SHT-036 is the administrative view of all Spaces and Workspaces the entity owns or is a member of. It provides an at-a-glance overview of the entity's full spatial presence on the platform.

Column	Type	Description
name	TextField	Space or Workspace display name
entity_type	EnumField	Space Workspace
space_type	EnumField	personal team org coop collective community federation project research event studio platform
namespace_path	TextField	Full KNSPC address for this Space/Workspace
status	EnumField	Draft Active Paused Restricted Dissolving Dissolved Archived
visibility	EnumField	Private Protected Internal Public Unlisted
member_count	NumberField	Active member count (Space only)
workspace_count	NumberField	Active Workspaces within this Space
my_role	EnumField	Owner Governor Steward Editor Contributor Member Viewer Guest
open_sheet_count	ComputedColumn	Number of sheets currently open in active Workspace session (Workspace rows only)
health_score	AIColumn	Space health: member activity, governance participation, treasury status, contribution velocity
link_count	LinkNetworkColumn	Inter-portfolio links involving this Space/Workspace
governance_model	EnumField	OwnerDecision AdminApproval MajorityVote SupermajorityVote Consensus MultiSig
treasury_balance	CurrencyField	Current treasury balance (kogi-bank sync). Blank for Workspaces and non-economic Spaces.
active_proposals	NumberField	Count of open governance proposals
last_activity	DateTimeField	Most recent EventLog entry timestamp for this Space/Workspace
federation_status	EnumField	None Federated PendingFederation — federation with other Spaces
namespace_aliases	TagField	List of alias paths that resolve to this entity
space_subgraph_size	ComputedColumn	Node count in this Space's SpaceLinkSubgraph

7.3 SHT-037: Namespace Directory Sheet

Column	Type	Description
path	TextField	Full NamespacePath string

Column	Type	Description
entity_type	EnumField	user org coop collective fed community project team workspace sheet portfolio component platform
slug	TextField	Terminal path segment
component_id	RelationField	Linked PortfolioComponent this path resolves to
visibility	EnumField	Public Protected Internal Private — KLNK and search discoverability
aliases	TagField	Alternative paths that redirect here
canonical_url	URLField	Full public URL for this namespace entry
federation_origin	TextField	Remote node ID if federated; blank for local
children_count	ComputedColumn	Count of child namespace entries
last_resolved_at	DateTimeField	Last successful path resolution
resolve_latency_ms	AIColumn	Average path resolution latency (kogi-engine telemetry)
health	EnumField	Active Stale Broken Redirected — namespace health status

8. Universal Column System — Complete Reference

8.1 Column Type Catalog

Column Type	Rust/Go Type	Description	Storage	Aggregations
TextField	String	Free text. Sortable, searchable. Markdown preview in tall row mode.	TEXT, FTS-indexed	COUNT, COUNT_NON_TY
NumberField	Decimal	Integer or decimal. Configurable decimal places.	NUMERIC(20,8)	SUM AVG MIN MEDIAN COUN
CurrencyField	Decimal+CurrencyCode	Decimal with ISO 4217 code. Multi-currency; auto-converts to base currency.	NUMERIC+VARCHAR(3)	SUM AVG MIN (base currency)
PercentField	f32	0-100. Progress bar in tall mode.	FLOAT4	AVG MIN MAX MEDIAN
DateField	NaiveDate	Calendar date. Timezone-aware display. Date math in formulas.	DATE, calendar-indexed	MIN MAX RAN
DateTimeField	DateTime<Utc>	Nanosecond-precision timestamp. Relative/absolute display modes.	TIMESTAMPTZ	MIN MAX RAN
DurationField	Duration(seconds)	Time span. "3d 4h" display. Arithmetic supported.	BIGINT (seconds)	SUM AVG MIN
EnumField	String (enum set)	One-of defined value set. Colored badge. Fast equality filter.	VARCHAR, hash-indexed	COUNT_BY_V MODE
MultiEnumField	Vec<String>	Set of enum values. Badge chips.	TEXT[] with GIN	COUNT_BY_V INTERSECTIO
RelationField	ComponentId (FK)	FK to another PortfolioRow. Linked name chip. Click-to-navigate.	UUID FK, join-resolved	COUNT COUNT_UNIQ
MultiRelationField	Vec<ComponentId>	Set of FK references. Chip cluster. Max 50.	UUID[] with GIN	COUNT COUNT_UNIQ
UserField	EntityId	User/org reference. Display name + avatar. @mention trigger.	UUID FK to entities	COUNT_UNIQ
MultiUserField	Vec<EntityId>	Set of user/org references. Avatar cluster.	UUID[] with GIN	COUNT_UNIQ LIST
TagField	Vec<String>	Tag chips. Autocomplete. Global taxonomy.	TEXT[] with GIN	UNION INTERSECTIO COUNT_BY_T
BoolField	bool	Checkbox. Tri-state in some contexts.	BOOLEAN	COUNT_TRUE COUNT_FALSE PCT_TRUE
RichTextField	String (Markdown)	Markdown rich text. Not sortable. Searchable.	TEXT, FTS-indexed	COUNT (non-e
FileRefField	FileId	File reference. Type icon + preview on hover.	UUID FK to files	COUNT

Column Type	Rust/Go Type	Description	Storage	Aggregations
URLField	String	Web URL. Clickable link with favicon.	TEXT	COUNT
JsonField	serde_json::Value	Arbitrary JSON. Raw inspector view. Used for complex nested data.	JSONB	COUNT
ComputedColumn	Dynamic (typed)	Derived from other columns. Read-only. Cached with TTL.	CellStore (Redis)	All numeric aggregations
AIColumn	Dynamic (typed)	Engine-generated signal. Read-only. Written by kogi-engine via WritebackService.	CellStore (Redis+PG)	AVG MIN MAX MEDIAN
FormulaColumn	Dynamic (typed)	User-defined formula via ColumnComputer expression language.	Formula stored; eval-time	All numeric aggregations
AuditColumn	Dynamic (String/int)	Derived from EventLog: last_actor, last_action, mutation_count.	Derived from EventLog	COUNT
AnalyticsColumn	u64 / f64	Aggregated engagement metric updated by AnalyticsEngine.	Materialized view in PG	SUM AVG MAX
LinkNetworkColumn	u32 / Vec<PortfolioId>	Computed from KLNK GraphEdge store.	Materialized from link_edges	SUM AVG MAX COUNT
IdentityColumn	Vec<IdentityTag>	Identity partition tags on this row. Determines profile membership.	TEXT[] with GIN	COUNT COUNT_BY_T
NamespaceColumn	NamespacePath	KNSPC address of this row's component.	VARCHAR, namespace-indexed	COUNT
SpaceColumn	ComponentId (Space FK)	Parent Space of this component.	UUID FK to components	COUNT COUNT_UNIQ
WorkspaceColumn	Vec<ComponentId>	Workspaces currently displaying this row.	UUID[] with GIN	COUNT COUNT_UNIQ

8.2 Space-Specific Column Groups

Components belonging to a Space carry additional column groups specific to their Space context:

Column Group	Columns	Available On Sheets
Space Identity	space_id · space_type · space_name · namespace_path · workspace_ids	SHT-036, SHT-001, SHT-023
Space Governance	governance_status · open_proposals · quorum_status · last_vote_at · multi_sig_required	SHT-019, SHT-018, SHT-036
Space Economics	treasury_balance · treasury_currency · revenue_ytd · expense_ytd · distribution_pending · last_distribution_at	SHT-018, SHT-009, SHT-036
Space Members	member_count · active_members · contribution_weights · governance_participants	SHT-019, SHT-036, SHT-021

Column Group	Columns	Available On Sheets
Space Link Graph	space_subgraph_size · space_external_links · space_federation_status · cross_space_projects	SHT-031, SHT-036
Workspace Session	workspace_name · session_active · last_session_at · open_sheets · active_board_mode · oba_state	SHT-038, SHT-036
Namespace	namespace_path · slug · canonical_url · alias_count · federation_origin · resolve_health	SHT-037, SHT-036

9. View Engine (KPVW) — Complete Specification

9.1 ViewDefinition — Full Schema

```
pub struct ViewDefinition {
  pub id:                ViewId,           // UUID
  pub name:              String,
  pub sheet_id:          SheetId,
  pub base_row_filter:   Option<ViewFilter>, // pre-filter before user
interaction
  pub column_schema:     ColumnSchema,
  pub filters:           Vec<ViewFilter>,
  pub sorts:             Vec<ViewSort>,
  pub groups:           Vec<ViewGroup>,
  pub row_height:        RowHeight,        // Compact | Normal | Tall | Auto
  pub highlight_rules:   Vec<RowHighlightRule>,
  pub pinned_columns:    Vec<ColumnId>,
  pub frozen_row_count:  u32,
  pub pivot_config:      Option<PivotConfig>,
  pub chart_config:      Option<ChartConfig>,
  pub summary_row:       Option<SummaryRowConfig>,
  pub board_config:      Option<BoardConfig>,
  pub link_network_overlay: bool,          // show link badges + KLNK panel
  pub identity_filter:   Option<IdentityTag>, // scope to partition
  pub space_filter:      Option<ComponentId>, // scope to Space
  pub workspace_filter:  Option<ComponentId>, // scope to Workspace
  pub namespace_filter:  Option<NamespacePath>, // scope to namespace subtree
  pub shadow_row_policy:  ShadowRowPolicy, // Include | Exclude | ShadowOnly
  pub visibility:        ViewVisibility, // Private | Shared | Public |
Template
  pub namespace_path:    NamespacePath,    // view's own KNSPC address
  pub created_by:        UserId,
  pub created_at:        DateTime<Utc>,
  pub updated_at:        DateTime<Utc>,
}

pub enum ShadowRowPolicy {
  Include,      // show own rows + ShadowRows interleaved
  Exclude,      // show only own rows
  ShadowOnly,   // show only ShadowRows from linked portfolios
}
```

9.2 Board Modes — Complete List

Board Mode	Description	Required Columns	Space Integration
Kanban	Swimlane columns = status values. Cards show configured fields. WIP limits per column. Drag-and-drop.	status, name, owners	Space-filtered view shows team's cards only
Agile	Sprint-scoped Kanban with story points, burndown overlay, velocity badge. DoR/DoD enforcement at column transitions.	status, story_points, sprint_ref	Shared Agile boards across Space team members
Gantt	Dependency-aware horizontal bars. Critical path highlighted. Resource bars overlaid. Milestone markers.	start_date, end_date/due_date	Space projects shown in swimlanes per team
Calendar	Day/week/month grid by due_date or start_date. Month/week/day views. Booking integration.	due_date or start_date	Space event calendar shared with all members
Resource Board	Rows = resources; columns = time periods; cells = allocation %. Red = over-allocated. AllocationEngine integration.	resource_type, allocated_hours	Space resource pool view
Network Graph	Force-directed node graph. Nodes = rows; edges = GraphEdges + KLNK edges. Color by type; size by health_score.	Any (all rows as nodes)	Space subgraph highlighted; external links in different color
Treemap	Hierarchical tiles sized by numeric column. Color by health_score. Drill-down. Budget/revenue composition.	Any numeric column for sizing	Space portfolio composition view
Timeline	Swimlane rows = programs/teams; bars = projects/milestones; nested. Quarter/month/week markers.	start_date, end_date	Space roadmap view
Link Forest	KLNK tree rendering. Nodes = portfolio components + linked external nodes; edges = LinkEdge types. Identity-colored branches.	link_count > 0	Space link subgraph rendered with member nodes highlighted
Identity Grid	Rows grouped by identity_tag. Columns = partition visibility settings. Color by isolation_level. For KPID management.	identity_tags, visibility	N/A (Personal Space only)
Space Map	Spatial rendering of all Spaces the entity is part of. Circles sized by member_count. Lines = federation edges.	space_id, member_count	Space-aware. External Space connections shown.
Namespace Tree	Hierarchical tree of KNSPC namespace entries. Path segments as collapsible nodes. Resolve status color-coded.	namespace_path	Space namespace subtree highlighted

9.3 Filter System — All Predicates

Predicate	Syntax	Column Types	Example
Equality	col = val	All	status = "Active"
Inequality	col != val	All	visibility != "Private"

Predicate	Syntax	Column Types	Example
Greater/Less	col > val / col < val	Number, Date	health_score > 70
Range	col BETWEEN a AND b	Number, Date	due_date BETWEEN 2026-04-01 AND 2026-06-30
Contains	col CONTAINS substr	Text, Tag, Rich	name CONTAINS "API"
In Set	col IN [v1, v2]	Enum, Tag, Relation	item_type IN ["Project", "Program"]
Not In Set	col NOT IN [v1, v2]	Enum, Tag, Relation	status NOT IN ["Archived", "Cancelled"]
Is Empty	col IS EMPTY	All nullable	due_date IS EMPTY
Is Not Empty	col IS NOT EMPTY	All nullable	owners IS NOT EMPTY
Tag Includes	col INCLUDES tag	Tag, MultiEnum	tags INCLUDES "q2-initiative"
User Is Me	col INCLUDES me	User, MultiUser	owners INCLUDES me
Identity Scoped	identity_tags INCLUDES @hdl	IdentityColumn	identity_tags INCLUDES "@jordan-dev"
Space Scoped	space_id = space_id	SpaceColumn	space_id = "pamoja-engineering-space-id"
Has Shadow	shadow_row_count > 0	LinkNetworkColumn	— shows only externally linked rows
Is Shadow	has_shadow_of IS NOT EMPTY	LinkMetadata	— shows only ShadowRows
Namespace Under	namespace_path STARTS "kogi://org/pamoja/"	NamespaceColumn	— all components under org namespace
Engine Score	health_score >= 80	AIColumn	risk_score > 50
Has Flag	risk_flags HAS_SEVERITY HIGH	JsonField/AIColumn	— components with high-severity risks
Date Relative	due_date < TODAY+7	DateField	— due within 7 days
AND Group	(f1 AND f2)	Composite	(status="Active" AND health_score < 60)
OR Group	(f1 OR f2)	Composite	(owned_by_me OR collaborating)
NOT	NOT f1	Composite	NOT (status IN ["Archived", "Cancelled"])
Subquery	id IN (SELECT id FROM rows WHERE ...)	All	id IN (SELECT id FROM rows WHERE link_count > 10)

10. Computation Engine (KPCM) — Complete Specification

10.1 Architecture

The Computation Engine operates on two tiers:

Tier	Description	Latency	Technology
Tier 1 — Synchronous	Simple arithmetic derived columns (budget_remaining, profit, roi, etc.). Evaluated in-request for every row returned by a sheet query. No caching required — always fresh.	<1ms per row	Rust ColumnComputer (embedded in spreadsheet-service)
Tier 2 — Asynchronous	Complex AI-driven signals (health_score, risk_score, match_score, income_projection, etc.). Computed by kogi-engine sub-engines. Results cached in CellStore. Invalidated on relevant EventLog events.	100ms–10s (batch); <500ms (on-demand)	Scala 3 kogi-engine + Go WritebackService + Redis CellStore

10.2 All 22 Computational Models

Model	Tier	Key Inputs	Key Outputs	Cache TTL
PortfolioHealth	2	child statuses, budget, risks, progress, engagement, collaboration_score, benefit_coverage	health_score 0-100, dimension breakdown, anomaly_list	30 min; invalidate on child status/budget change
ProjectMetrics	2	sprint_velocity_history, backlog_size, completion_rate, cycle_time, lead_time, blocker_count	completion_pct, velocity, predicted_completion_date, risk_level	5 min; invalidate on sprint/task update
ProgramAlignment	2	child project health scores, KPI actuals vs targets, budget rollup, milestone status	alignment_score 0-100, kpi_achievement_rate	1 hr; invalidate on child project health change
SubPortfolioRollup	2	all descendant component financial and status data (recursive traversal)	rollup_score 0-100, aggregated_financials	1 hr; invalidate on descendant change
ResourceUtilisation	2	allocation records, capacity units, consumed units, schedule overlap	utilisation_pct, demand_ratio, over_committed, conflict_list	15 min; invalidate on allocation change
AssetValue	2	asset type, acquisition cost, valuation history, market data, depreciation schedule	current_value, depreciation, roi_pct, irr	1 hr; invalidate on valuation event

Model	Tier	Key Inputs	Key Outputs	Cache TTL
ArtifactMaturity	2	artifact type, version count, review status, project completion, documentation completeness	maturity_score 0-100, completeness_breakdown	2 hr; invalidate on artifact update
CollaborationScore	2	contributor count, contribution velocity, governance participation, merge conflict rate	collaboration_score 0-100	30 min; invalidate on contribution event
BenefitsHealth	2	benefit balances, coverage types, income replacement ratio, vesting progress	benefits_health_score 0-100, coverage_gap_list	6 hr; invalidate on benefit event
RiskScore	2	risk_flags (severity+probability), mitigation status, overdue items, blocker chain depth	risk_score 0-100, top_risk_items, mitigation_gap_flags	15 min; invalidate on risk_flag change
IncomeProjection	2	gig earnings 90d, active contract revenue, investment distributions, seasonal patterns	projected_income_{30,60,90}d with CI, diversification_score	6 hr; invalidate on income event
CampaignHealth	2	contribution velocity, time-to-deadline, backers/day, social graph engagement	campaign_health_score, success_probability, days_to_goal	5 min for active campaigns; 1 hr for inactive
GrantMatchScore	2	eligibility criteria match, portfolio strength, historical award data for similar portfolios	grant_match_score 0-100, eligibility_flags	24 hr; invalidate on portfolio composition change
LinkNetworkStrength	2	link_count, inbound_count, link_diversity, mutual_connections, network centrality	network_strength_score 0-100, centrality_rank	1 hr; invalidate on LinkEdge change
SpaceHealth	2	member activity, governance participation, treasury status, contribution velocity, federation	space_health_score 0-100, at_risk_members, governance_health	1 hr; invalidate on Space event
WorkspaceUtilisation	2	session frequency, open_sheet_count, active_board_mode history, Oba interaction rate	workspace_utilisation_score, most_used_sheets, idle_time	6 hr; per-session update
NamespaceHealth	2	last_resolved_at, resolve_latency, broken_aliases, stale_entries, federation_sync_status	namespace_health_score, stale_paths, broken_paths	1 hr; invalidate on namespace event
MarketabilityScore	2	listing quality, pricing competitiveness vs PriceEngine, CTR/conversion rate	marketability_score, listing_optimization_hints	2 hr; invalidate on listing update
IdentityCoverage	2	ProfilePartition completeness: required fields, visibility mask, public content present	identity_coverage_score per partition, missing_fields	1 hr; invalidate on partition update
BudgetRemaining	1	budget_allocated, budget_spent	budget_remaining, budget_utilization_pct	N/A (sync tier)
Profit	1	revenue, expenses	profit, profit_margin_pct	N/A (sync tier)

Model	Tier	Key Inputs	Key Outputs	Cache TTL
ROI	1	revenue, budget_spent	roi_pct	N/A (sync tier)

10.3 ColumnComputer — Formula Language Reference

Category	Functions	Return Type	Notes
Arithmetic	+ - × ÷ % ^ ABS(x) ROUND(x,n) CEIL(x) FLOOR(x) MIN(a,b) MAX(a,b)	Number/Currency	Currency-aware: mixed-currency arithmetic auto-converts to base currency
Comparison	= != > < >= <=	Bool	
Logical	AND OR NOT IF(cond,t,f) SWITCH(val,case:result,...) COALESCE(a,b,...)	Dynamic	COALESCE: returns first non-null value
String	CONCAT(...) UPPER(s) LOWER(s) LEN(s) TRIM(s) LEFT(s,n) RIGHT(s,n) SUBSTR(s,i,n) REPLACE(s,f,r) CONTAINS(s,q) STARTS(s,p)	String	
Date	TODAY() NOW() YEAR(d) MONTH(d) DAY(d) QUARTER(d) WEEKNUM(d) DAYNAME(d) DAYS_BETWEEN(a,b) DATE_ADD(d,n,unit) DATE_FORMAT(d,fmt)	Date/Number/String	unit: day week month year
Aggregation	SUM(field) AVG(field) COUNT(field) MIN(field) MAX(field) MEDIAN(field) STDDEV(field) PERCENTILE(field,p)	Number	Over related children: SUM(CHILDREN.budget_spent)
Lookup	RELATED(rel_col, target_col) LOOKUP(key, src_col, tgt_col) CORELATED(rel_col, filter, tgt_col)	Dynamic	RELATED: dereferences a RelationField and reads target_col from the related row
Space	SPACE_NAME() SPACE_TYPE() SPACE_MEMBER_COUNT() IN_SPACE(space_id)	String/Bool	Returns Space context of the current row
Identity	IDENTITY_TAG() IS_IN_PARTITION(tag) VISIBLE_TO(observer_type)	String/Bool	observer_type: Public Follower Connection Member Owner
Link Network	LINK_COUNT() INBOUND_LINKS() OUTBOUND_LINKS() IS_LINKED_TO(portfolio_id) LINK_DEPTH()	Number/Bool	
Namespace	NAMESPACE_PATH() NAMESPACE_SLUG()	String/Bool	

Category	Functions	Return Type	Notes
	NAMESPACE_SCOPE() IS_UNDER_NAMESPACE(path)		
Engine	HEALTH_SCORE() RISK_SCORE() MATCH_SCORE() INCOME_PROJECTION(days) SPACE_HEALTH()	Number	Injects Tier-2 engine values as formula operands
Conditional Fmt	HIGHLIGHT(condition, color_hex) FLAG(condition, label) BADGE(value, color_map)	DisplayHint	Returns display hint only; does not affect data

11. Collaborative Editing & CRDT (KPCL)

11.1 CRDT Operation Types — Complete Reference

Operation	Type	Semantics	Identity/Space Context
SetField	LWW	Last-Write-Wins field update. Accepts remote op only if remote timestamp > local. Tiebreak by last_actor string sort.	actor_identity_tag in op. Cross-partition writes checked against SplitPolicy.
AddToSet	OR-Set	Add a value to a set column (tags, owners, policy_ids, workspace_ids, identity_tags). Both concurrent adds survive.	identity_tags additions require write permission on target partition.
RemoveFromSet	OR-Set	Remove a specifically tagged entry from a set column. OR-Set remove: only removes the exactly-tagged item.	identity_tags removals audit-logged as PartitionTagEvent.
AppendLog	Append	Append an EventLog entry. Commutative — all concurrent appends survive. Ordered by VectorClock causal timestamp.	EventLog entries carry identity_tag. Cross-partition entries clearly marked.
AttachChild	OR-Set	Add a GraphEdge::Hierarchy edge. Concurrent attaches both survive.	Cross-partition hierarchy edges require owner approval if target partition is HardIsolated.
DetachChild	OR-Set	Remove a GraphEdge::Hierarchy edge. Resolves to higher-timestamp when concurrent with Attach.	Same as AttachChild.
StatusTransition	Lattice	Status uses a lifecycle lattice (§4.5), not LWW. Merge to least upper bound. PolicyEngine validates before accept.	Shared portfolio transitions may require GovernanceVote depending on SpaceGovernanceConfig.
CreateShadowRow	Special	Create a ShadowRow in target spreadsheet on KLNK edge confirmation. One-time operation.	Requires valid LinkEdge with consent_status=Accepted. Space-scoped LinkEdges create Space-scoped ShadowRows.
SyncMirrorColumn	LWW	Update MirrorColumn value from source. Last-sync-wins. Blocked if source LinkEdge is Disconnected.	Source identity must still have active LinkEdge. Revoked links block sync.
SpaceRoleChange	LWW	Update a SpaceMember's role. Checked against Space governance rules. GovernanceVote may be required.	Space-scoped. Requires Steward+ in the Space.
NamespaceUpdate	LWW	Update a NamespacePath for a component. Alias created for old path. Cascade updates to child namespace entries.	Owner-only. Triggers async namespace-service migration.
WorkspaceUpdate	LWW	Update Workspace state: open_sheets, pinned_rows, layout_config, etc.	User-scoped. Shared Workspaces: LWW with presence broadcasting.

11.2 VectorClock Implementation

```
pub struct VectorClock {
    pub clocks: HashMap<NodeId, u64>, // NodeId = federation node identifier
}

impl VectorClock {
    pub fn tick(&mut self, node_id: &NodeId) -> u64 {
        let counter = self.clocks.entry(node_id.clone()).or_insert(0);
        *counter += 1;
        *counter
    }

    pub fn merge(&mut self, other: &VectorClock) {
        for (node, &remote_ts) in &other.clocks {
            let local_ts = self.clocks.entry(node.clone()).or_insert(0);
            *local_ts = (*local_ts).max(remote_ts);
        }
    }

    pub fn happened_before(&self, other: &VectorClock) -> bool {
        self.clocks.iter().all(|(n, &ts)| {
            other.clocks.get(n).copied().unwrap_or(0) >= ts
        }) && self.clocks != other.clocks
    }

    pub fn concurrent_with(&self, other: &VectorClock) -> bool {
        !self.happened_before(other) && !other.happened_before(self)
    }
}

// NodeId for identity-aware CRDT: "{federation_node}:{identity_tag}"
// E.g. "node-us-east-1:@jordan-dev"
// This ensures identity-tagged operations carry the correct causal context
```

11.3 Federation Protocol

Phase	Description	Services Involved	Data Exchanged
Discovery	A PortfolioFederation registers a new peer. Peer endpoint URL validated. Trust flag set by admin.	portfolio-service, namespace-service	FederationPeer record; initial VectorClock state

Phase	Description	Services Involved	Data Exchanged
Handshake	Both nodes exchange current VectorClock states and last_synced timestamps to compute delta to sync.	crdt-service	VectorClock exchange; last_synced timestamp
Delta Compute	Compute the set of CrdtLog operations since the peer's last known VectorClock. Can be large for long-offline nodes.	crdt-service	Filtered CrdtLog batch (operations since peer's last VectorClock)
Delta Push	Send the delta CrdtLog batch to the peer. Peer applies operations via apply_crdt_log().	crdt-service (both nodes)	CrdtLog batch: Vec<CrdtOperation>
Conflict Merge	Concurrent operations are identified and merged per CRDT type. Conflicts surfaced as ConflictRecord for human review.	crdt-service	ConflictRecord list; merged component state
Namespace Sync	Namespace registry entries for federated components propagated to both nodes' NamespaceRegistries.	namespace-service	NamespaceRegistry entries for federated paths
Link Sync	KLNK LinkEdges involving federated components replicated to both nodes' link_edges tables.	link-network-service	LinkEdge records for cross-node edges
Heartbeat	Regular (every 30s) lightweight sync check. Exchange only VectorClock state. Full delta sync if divergence detected.	crdt-service	VectorClock state only

12. kogi-engine Integration — Complete Reference

The kogi-engine is the Scala 3 intelligence substrate that consumes all portfolio events via gRPC and Kafka, computes analytical signals, and writes them back into portfolio rows via the WritebackService. All kogi-engine sub-systems are accessed internally via gRPC on port 9100.

12.1 Engine Sub-System Registry

Engine	Scala Class	Primary Role in KIMDSS	Output Columns
AnalyticsEngine	AnalyticsEngine.scala	Computes engagement, performance, and usage analytics for all PortfolioComponents.	views, clicks, ctr, engagement_rate, followers, spread, sentiment_score
TelemetryEngine	TelemetryEngine.scala	Real-time event streaming: component state changes, user activity, mutation velocity.	event_count, mutation_velocity, last_event_at, flow_stats
RecommendationEngine	RecommendationEngine.scala	Personalized content and opportunity discovery. Collaborative + content-based hybrid.	recommendation_score, similar_items, suggested_actions
PersonalizationEngine	PersonalizationEngine.scala	Persona lifecycle, A/B experiments, multi-armed bandit for content variants.	personalized_rank, relevance_score, persona_label
MatchEngine	MatchEngine.scala	Multi-dimensional scoring to match entities: worker-opportunity, investor-campaign, resource-project.	match_score, talent_match_score, investor_match_score
GraphEngine	GraphEngine.scala	Dependency graph traversal, critical path analysis, cycle detection, impact analysis.	dependency_depth, critical_path_flag, network centrality
RiskEngine	RiskEngine.scala	Risk scoring: aggregates risk_flags with probability weighting. Produces OptimizationPlan.	risk_score, risk_breakdown, mitigation_gap_flags, at_risk_deadline_count
OptimizationEngine	OptimizationEngine.scala	Monte Carlo forecasting, bottleneck detection, resource allocation recommendations.	predicted_completion_date, schedule_risk_flag, budget_risk_flag

Engine	Scala Class	Primary Role in KIMDSS	Output Columns
SearchEngine	SearchEngine.scala	Full-text + semantic search indexing and ranked retrieval across all PortfolioComponents.	search_rank, full_text_index_status, keyword_suggestions
AllocationEngine	AllocationEngine.scala	Human and physical resource allocation; capacity planning; over-allocation detection.	capacity_conflicts, allocation_recommendations, utilization_heatmap
IncentiveEngine	IncentiveEngine.scala	KogiPoints accounting, reputation scoring, streak tracking, badge management.	kogipoints_earned, incentive_streak, reputation_tier, badges
CollaborationEngine	CollaborationEngine.scala	Shared portfolio coordination, contribution attribution, concurrent edit conflict scoring.	collaboration_score, contributor_diversity_index, merge_conflict_count
BenefitsEngine	BenefitsEngine.scala	Benefits eligibility scoring, coverage optimization, gap analysis, contribution forecasting.	benefits_health_score, coverage_gap_flags, tax_savings_estimate
GrantEngine	GrantEngine.scala	Grant matching by eligibility, portfolio strength, and historical award data.	grant_match_score, eligibility_flags, recommended_grants
CrowdfundingEngine	CrowdfundingEngine.scala	Campaign health monitoring, success probability, investor match quality.	campaign_health_score, success_probability, investor_match_quality
BookingEngine	BookingEngine.scala	Scheduling optimization, availability conflict detection, booking revenue forecasting.	availability_conflicts, booking_revenue_forecast, utilization_calendar
CRMEngine	CRMEngine.scala	Lead scoring, pipeline value estimation, follow-up due date calculation.	lead_score, pipeline_value, follow_up_due_flags
LogisticsEngine	LogisticsEngine.scala	Physical resource routing, equipment tracking, crew schedule optimization.	logistics_status, itinerary_health, resource_routing_conflicts

Engine	Scala Class	Primary Role in KIMDSS	Output Columns
SpaceHealthEngine	SpaceHealthEngine.scala	Space-specific health: member activity rates, governance participation, treasury status.	space_health_score, at_risk_members, governance_health
NamespaceEngine	NamespaceEngine.scala	Namespace health: stale paths, broken aliases, resolve latency, federation sync status.	namespace_health_score, stale_paths, broken_paths
DataStreamingEngine	DataStreamingEngine.scala	Real-time data export, BI connector feeds, event bus subscriptions.	N/A (infrastructure layer)

12.2 WritebackService Protocol

```
// WritebackService gRPC interface
service WritebackService {
    rpc WriteColumn(WriteColumnRequest) returns (WriteColumnResponse);
    rpc WriteBatch(WriteColumnBatchRequest) returns (WriteColumnBatchResponse);
    rpc WriteAnomalyFlags(WriteAnomalyRequest) returns (WriteAnomalyResponse);
    rpc WriteObaHint(WriteObaHintRequest) returns (WriteObaHintResponse);
}

message WriteColumnRequest {
    string component_id      = 1;
    string column_id         = 2;
    google.protobuf.Value value = 3; // typed per column definition
    float confidence         = 4; // 0.0-1.0
    string source_engine     = 5; // e.g. "RiskEngine"
    int64 computation_ts     = 6; // nanosecond epoch
    int64 ttl_seconds        = 7; // CellStore cache TTL
    string space_id          = 8; // optional Space scope
    string identity_tag       = 9; // partition scope if applicable
}

// WritebackService pipeline:
// 1. Validate source_engine has AIWriter permission for column_id
// 2. Evaluate PolicyEngine for any blocking policies on the write
// 3. If column crosses alert threshold: dispatch KNTF notification
// 4. Write value to CellStore with ttl_seconds
// 5. Append PortfolioEventKind::AIColumnUpdated to component EventLog
// 6. WebSocket broadcast CellUpdateEvent to all Workspace subscribers
// 7. If Space-scoped: broadcast to all Space member Workspaces viewing this sheet
```


13. Persistence & Storage Architecture

13.1 Storage Layer — Complete Reference

Store	Technology	Tables / Collections	Access Pattern	Backup
Primary Component Store	PostgreSQL 16 + JSONB	components (JSONB data+metadata), graph_edges, event_log (partitioned monthly), resource_allocations	Read-heavy; JSONB-indexed; partitioned by owner_entity_id	Continuous WAL streaming to S3
Space/Workspace Store	PostgreSQL 16	spaces, workspaces, space_members, workspace_sessions, governance_proposals, treasury_accounts	Read: on auth/session; Write: on Space events	Same as PG
Namespace Registry	PostgreSQL 16	namespace_registry (path, component_id, entity_type, aliases[], federation_origin)	Read: every request (Redis-cached 300s); Write: on rename/create	Same as PG
KLNK Link Store	PostgreSQL 16	link_nodes, link_edges, shadow_rows, mirror_columns, link_subscriptions, space_link_subgraphs (materialized)	Read: graph traversal queries; Write: on link create/delete	Same as PG
Identity Store	PostgreSQL 16	profile_partitions, visibility_masks, account_partition_mappings, identity_anchors, split_policies	Read: every authenticated request; Write: on identity ops	Same as PG
CRDT Buffer	Redis Cluster	crdt_log:{component_id} (hash), vector_clock:{component_id} (hash), cell_cache:{component_id}:{column_id} (string)	Sub-millisecond read/write; TTL expiry; flushed to PG	Redis RDB snapshots every 15m to S3
Cell Cache	Redis Cluster	cell:{component_id}:{column_id} = {value, confidence, source_engine, computed_at}	Read: O(1) per cell; Write: on engine writeback; TTL invalidation	Same as PG
Workspace Session Cache	Redis Cluster	ws_session:{workspace_id}:{user_id}, cursor:{workspace_id}:{user_id}, presence:{space_id}	Real-time; TTL-based; session heartbeat	Not persisted (ephemeral)
Namespace Resolution Cache	Redis Cluster	ns_resolve:{path} = {component_id, visibility, last_resolved}	Read: O(1) with 300s TTL; Write: on namespace change	Not critical (rebuilt from PostgreSQL)
Full-Text Search Index	Meilisearch / Typesense	component index (name, description, tags, properties, namespace_path, identity_tags, space_id)	Read: search queries (ms latency); Write: async on component create/update	Rebuilt from PostgreSQL
Analytics Store	ClickHouse	component_events (append-only: component_id, event_type, actor, timestamp, delta)	Append-only writes; aggregate reads; never updated	Continuous replication to S3 Parquet

Store	Technology	Tables / Collections	Access Pattern	Backup
Engine Feature Store	Redis + PostgreSQL MV	engine_features:{component_id} (Redis hash), materialized views for health_score, risk_score, match_score per type	Read: per-component on sheet load; Write: engine writeback	PostgreSQL MV rebuild hourly for events
Shadow Row Store	PostgreSQL 16	shadow_rows, mirror_columns, shadow_sync_log, consent_requests	Read: on sheet load (host user); Write: on link create + sync	Same as Primary
Snapshot Store	S3-compatible (MinIO / AWS S3)	snapshots/{entity_id}/{date}.snap.json.gz, checkpoints/{entity_id}/{label}.chk.json.gz	Write: scheduled + manual; Read: on restore; lifecycle-managed	S3 versioning, cross-region replication
Federation Peer Store	PostgreSQL 16	federation_peers, federation_sync_log, federated_namespace_entries	Read: on sync operations; Write: on peer events	Same as Primary

13.2 PostgreSQL Schema — Core Tables

```
-- Primary component table
CREATE TABLE components (
  id                UUID PRIMARY KEY,
  owner_entity_id   UUID NOT NULL,          -- partitioned by this
  item_type         VARCHAR(64),
  container_type     VARCHAR(64),
  status            VARCHAR(32) NOT NULL DEFAULT 'Draft',
  visibility         VARCHAR(32) NOT NULL DEFAULT 'Private',
  space_id          UUID REFERENCES components(id),
  namespace_path     VARCHAR(512),
  identity_tags      TEXT[],                -- GIN indexed
  tags              TEXT[],                -- GIN indexed
  metadata          JSONB NOT NULL DEFAULT '{}', -- ComponentMetadata
  data              JSONB NOT NULL DEFAULT '{}', -- ComponentData
  health_score       FLOAT4,                -- cached from engine
  risk_score        FLOAT4,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY HASH (owner_entity_id);

CREATE INDEX idx_components_owner ON components (owner_entity_id, status, updated_at DESC);
CREATE INDEX idx_components_type ON components (item_type, status, visibility);
CREATE INDEX idx_components_space ON components (space_id, status);
CREATE INDEX idx_components_tags ON components USING GIN (tags);
CREATE INDEX idx_components_itags ON components USING GIN (identity_tags);
CREATE INDEX idx_components_ns ON components (namespace_path);
```

```

CREATE INDEX idx_components_scores ON components (health_score DESC, risk_score
DESC);

-- Graph edges (Hierarchy, Dependency, Link, InterPortfolioLink, etc.)
CREATE TABLE graph_edges (
    edge_id          UUID PRIMARY KEY,
    source_id        UUID NOT NULL,
    target_id        UUID NOT NULL,
    edge_type        VARCHAR(64) NOT NULL,
    weight           FLOAT4 DEFAULT 1.0,
    metadata         JSONB DEFAULT '{}',
    space_id         UUID,                -- Space scope (NULL = global)
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_edges_source ON graph_edges (source_id, edge_type);
CREATE INDEX idx_edges_target ON graph_edges (target_id, edge_type);

-- Event log (append-only, monthly partitioned)
CREATE TABLE event_log (
    entry_id         UUID DEFAULT gen_random_uuid(),
    component_id     UUID NOT NULL,
    actor_id         VARCHAR(128) NOT NULL,    -- "{user_id}:{identity_tag}"
    event_type       VARCHAR(64) NOT NULL,
    delta            JSONB NOT NULL,
    vector_clock     JSONB NOT NULL,
    space_id         UUID,
    workspace_id     UUID,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (created_at);
-- Monthly partitions: event_log_2026_01, event_log_2026_02, ...

-- Namespace registry
CREATE TABLE namespace_registry (
    namespace_id     UUID PRIMARY KEY,
    path             VARCHAR(512) UNIQUE NOT NULL,
    component_id     UUID NOT NULL REFERENCES components(id),
    entity_type      VARCHAR(64) NOT NULL,
    slug             VARCHAR(128) NOT NULL,
    parent_namespace_id UUID REFERENCES namespace_registry(namespace_id),
    visibility       VARCHAR(32) NOT NULL DEFAULT 'Public',
    aliases          TEXT[] DEFAULT '{}',
    canonical_url    VARCHAR(1024),
    federation_origin VARCHAR(128),
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_resolved_at TIMESTAMPTZ

```

```
);  
CREATE INDEX idx_ns_path      ON namespace_registry (path);  
CREATE INDEX idx_ns_parent    ON namespace_registry (parent_namespace_id);  
CREATE INDEX idx_ns_slug      ON namespace_registry (slug, entity_type);
```

14. Security, Access Control & Compliance

14.1 Authentication & Authorization Flow

```
// Every request to any KIMDSS service follows this pipeline:

1. api-gateway: validate JWT / API key / session token
   → extract: sovereign_entity_id, active_identity_tag, active_space_id,
   active_workspace_id
   → rate limit check (per entity + per endpoint)

2. identity-service: resolve ProfilePartition for active_identity_tag
   → load VisibilityMask for the partition
   → check IsolationLevel if cross-partition access attempted

3. portfolio-service / space-service: PermissionTier check
   → evaluate ComponentData.users for the caller's tier on the target component
   → tier: Viewer(1) Subscriber(2) Contributor(3) Editor(4) Manager(5) Owner(6)
   Admin(7)

4. PolicyEngine: evaluate all attached policy_ids
   → PolicyDecision: Allow | Deny(reason) | RequireApproval(reason)
   → SplitPolicy: check cross-partition rules

5. Row-Level Security (PostgreSQL RLS):
   → WHERE clause injected: visibility IN (allowed_set) OR owner_entity_id = caller
   → identity_tags filter applied: identity_tags && ARRAY[visible_partitions]
   → space_id filter applied if space-scoped request

6. Column-Level Masking (application layer):
   → VisibilityMask applied to result rows
   → Owner-only columns stripped for non-owner callers
   → ShadowRows: only visible_columns returned (mirror_columns only)
```

14.2 Permission Matrix

Action	View er	Subscr iber	Contri butor	Edit or	Man ager	Own er	Adm in
Read own components	✓	✓	✓	✓	✓	✓	✓
Read public components	✓	✓	✓	✓	✓	✓	✓
Read follower-visible components	✗	✓	✓	✓	✓	✓	✓

Action	View er	Subscr iber	Contri butor	Edit or	Man ager	Own er	Adm in
Follow / Subscribe / Watch	X	✓	✓	✓	✓	✓	✓
Create KLNK link request	X	✓	✓	✓	✓	✓	✓
Submit contribution to shared	X	X	✓	✓	✓	✓	✓
Edit component fields	X	X	X	✓	✓	✓	✓
Create child components	X	X	X	✓	✓	✓	✓
Add custom columns	X	X	X	✓	✓	✓	✓
Create/manage Workspaces	X	X	X	✓	✓	✓	✓
Manage Space settings	X	X	X	X	✓	✓	✓
Manage Space members	X	X	X	X	✓	✓	✓
Submit governance proposals	X	X	X	X	✓	✓	✓
Manage treasury operations	X	X	X	X	✓	✓	✓
Configure identity partitions	X	X	X	X	X	✓	✓
Delete/archive components	X	X	X	X	X	✓	✓
CrossIdentityMerge	X	X	X	X	X	✓	✓
Dissolve Space	X	X	X	X	X	✓	✓
Platform-level admin override	X	X	X	X	X	X	✓

14.3 Data Encryption

Data Category	Encryption Method	Key Management	Notes
All data in transit	TLS 1.3	Platform-managed certificates (Let's Encrypt / ACM)	All services; mutual TLS for internal gRPC
All data at rest (PG)	AES-256 (transparent)	PostgreSQL TDE via pgcrypto; keys in AWS KMS / Vault	Storage-level encryption
PII fields	Field-level AES-256	Per-Sovereign-Entity key derivation from master key in Vault	legal_name, phone, email, gov_id, bank_account_ref
Health data (benefit)	Field-level AES-256	Same as PII	BenefitAccount balances, claim details, HSA transactions
HardIsolated partition rows	Field-level AES-256 (extra)	Per-partition key derived from per-entity key	Rows in HardIsolated partitions encrypted with partition-specific key
DM / E2E messages	AES-256 per-conversation	Key managed by kogi-host; not accessible to platform	Optional. Community messaging layer.

Data Category	Encryption Method	Key Management	Notes
Snapshots (S3)	AES-256 SSE-S3	S3-managed or KMS-managed	Signed with HMAC for integrity verification
GDPR export archives	AES-256 + signed	User-provided passphrase (optional) + platform signing key	Recipient-verifiable integrity

15. API Surface — Complete Reference

15.1 Portfolio API

Endpoint	Method	Description	Min Auth
/portfolio/:id	GET	Read component by ID	Viewer
/portfolio/@:handle/:slug	GET	Read component by identity handle + slug	Viewer
/portfolio	POST	Create component (item or container)	Contributor
/portfolio/:id	PATCH	Update component fields	Editor
/portfolio/:id	DELETE	Soft-archive component	Owner
/portfolio/:id/children	GET	List direct children	Viewer
/portfolio/:id/descendants	GET	All transitive descendants (paginated)	Viewer
/portfolio/:id/attach/:child_id	POST	Attach child (Hierarchy edge)	Editor
/portfolio/:id/attach/:child_id	DELETE	Detach child	Editor
/portfolio/:id/actions	POST	ActionKind dispatch: like/share/follow/invest/donate/tag	Contributor
/portfolio/:id/health	GET	PortfolioHealth model output	Viewer
/portfolio/:id/eventlog	GET	EventLog entries (paginated)	Editor
/portfolio/:id/allocate	POST	Create ResourceAllocation	Manager
/portfolio/:id/record-spend	POST	Record budget spend	Editor
/portfolio/search	GET	Full-text + faceted search	Viewer
/portfolio/query	POST	PQL query endpoint	Editor
/portfolio/:id/snapshot	POST	Create named snapshot	Owner
/portfolio/restore/:snapshot_id	POST	Restore from snapshot	Owner
/portfolio/export/:sheet_id	GET	Export sheet (CSV/XLSX/JSON/PDF)	Editor
/portfolio/:id/contribute	POST	Submit contribution to shared component	Contributor
/portfolio/:id/crdt/sync	POST	Submit CrdtLog batch (federation peers)	System

15.2 Space & Workspace API

Endpoint	Method	Description	Min Auth
/spaces	GET	List all Spaces the caller is member of	Viewer

Endpoint	Method	Description	Min Auth
/spaces	POST	Create a new Space	Contributor
/spaces/:id	GET	Get Space details and members	Viewer
/spaces/:id	PATCH	Update Space configuration	Manager
/spaces/:id/join	POST	Join a Space (public) or accept invitation	Contributor
/spaces/:id/members	GET	List Space members with roles	Member
/spaces/:id/members/:user_id	PATCH	Update member role	Steward
/spaces/:id/members/:user_id	DELETE	Remove member from Space	Steward
/spaces/:id/workspaces	GET	List Workspaces within this Space	Member
/spaces/:id/governance/proposals	GET	List governance proposals	Member
/spaces/:id/governance/propose	POST	Submit governance proposal	Governor
/spaces/:id/governance/vote	POST	Cast vote on a proposal	Member
/spaces/:id/treasury	GET	Get treasury account status	Member
/spaces/:id/namespace	GET	Get all namespace entries under this Space	Member
/spaces/:id/link-subgraph	GET	Get materialized SpaceLinkSubgraph	Member
/spaces/:id/health	GET	SpaceHealth model output	Member
/spaces/:id/dissolve	POST	Initiate dissolution (requires governance vote)	Owner
/workspaces	POST	Create a new Workspace	Editor
/workspaces/:id	GET	Get Workspace state	Member
/workspaces/:id	PATCH	Update Workspace configuration (sheets, layout)	Owner(ws)
/workspaces/:id/session	POST	Open a Workspace session (returns WebSocket URL)	Member
/workspaces/:id/session	DELETE	Close active Workspace session	Member
/workspaces/:id/share	POST	Share Workspace with another user	Owner(ws)
/workspaces/:id/history	GET	Get session history for this Workspace	Owner(ws)

15.3 Namespace API

Endpoint	Method	Description	Min Auth
/namespace/resolve	GET	Resolve a NamespacePath to a ComponentId	Viewer

Endpoint	Method	Description	Min Auth
/namespace	POST	Register a new namespace entry	Owner(parent ns)
/namespace/:id	PATCH	Update namespace entry (rename, alias)	Owner(ns)
/namespace/:id	DELETE	Delete namespace entry (only if archived)	Owner(ns)
/namespace/:id/alias	POST	Add a path alias	Owner(ns)
/namespace/:id/alias/:alias	DELETE	Remove a path alias	Owner(ns)
/namespace/:id/children	GET	List child namespace entries	Viewer
/namespace/health	GET	Get namespace health report (stale, broken, federation)	Admin
/namespace/:id/federate	POST	Publish namespace entry to federation peer	Federation Admin

15.4 Portfolio Query Language (PQL) — Full Syntax

```
-- PQL Grammar (EBNF)

query      ::= SELECT cols FROM "rows" where? groupby? orderby? limit? options?
cols       ::= "*" | col ("," col)*
col        ::= col_name | agg_fn "(" col_name ")" | formula_expr
where      ::= "WHERE" cond
cond       ::= pred | "(" cond "AND" cond ")" | "(" cond "OR" cond ")" | "NOT" cond
pred       ::= col_name op val | col_name "IS" "EMPTY" | col_name "IS NOT" "EMPTY"
            | col_name "INCLUDES" val | col_name "HAS_SEVERITY" sev
groupby    ::= "GROUP BY" col_name ("," col_name)* rollup?
rollup     ::= "," "ROLLUP" "(" col_name ")"
orderby    ::= "ORDER BY" col_name dir? ("," col_name dir?)*
dir        ::= "ASC" | "DESC"
limit      ::= "LIMIT" int ("OFFSET" int)?

-- Options (PQL extensions)
options    ::= identity_scope? space_scope? shadow? as_of?
identity_scope ::= "IDENTITY" string_lit      -- filter to identity partition
space_scope   ::= "IN SPACE" string_lit       -- filter to Space
shadow        ::= "INCLUDE SHADOW ROWS"       -- include ShadowRows in results
            | "SHADOW FROM" string_lit       -- ShadowRows from specific user
            | "SHADOW ONLY"                 -- only ShadowRows
as_of         ::= "AS_OF" timestamp          -- time-travel query
namespace    ::= "UNDER" string_lit          -- filter to namespace subtree
```

```
-- Examples
SELECT id, name, health_score, budget_remaining, namespace_path
FROM rows
WHERE item_type = "Project"
      AND status = "Active"
      AND health_score < 70
IN SPACE "kogi://org/pamoja-capital/"
ORDER BY due_date ASC NULLS LAST
LIMIT 50;

SELECT space_id, SUM(budget_allocated), SUM(budget_spent), AVG(health_score)
FROM rows
WHERE item_type IN ("Project", "Program")
      AND quarter = "Q2-2026"
GROUP BY space_id
ORDER BY SUM(budget_spent) DESC;

SELECT id, name, benefit_type, balance, coverage_gap_flag
FROM rows
WHERE item_type = "BenefitAccount"
      AND coverage_gap_flag IS NOT EMPTY
IDENTITY "@jordan-dev"
ORDER BY benefit_type;

-- Cross-Space with ShadowRows
SELECT name, status, health_score, source_owner_handle
FROM rows
WHERE item_type = "Project"
INCLUDE SHADOW ROWS
ORDER BY health_score DESC;

-- Time-travel
SELECT name, status, budget_remaining
FROM rows
WHERE item_type = "Portfolio"
AS_OF "2026-01-01T00:00:00Z";
```

16. Error Handling, Resilience & Observability

16.1 Error Model

```
pub enum KIMDSSError {
  // Component errors
  NotFound          { component_id: ComponentId },
  PermissionDenied  { user_id: String, action: String, required_tier:
PermissionTier },
  InvalidOperation  { message: String },
  CyclicDependency  { source: ComponentId, target: ComponentId },
  VersionConflict   { local: VersionString, remote: VersionString },
  AlreadyExists     { component_id: ComponentId },
  InvalidState      { current: ComponentState, attempted: String },
  StorageError      { message: String },
  ValidationError   { field: String, message: String },

  // Space/Workspace errors
  SpaceNotFound     { space_id: ComponentId },
  NotSpaceMember    { space_id: ComponentId, user_id: String },
  GovernanceRequired { action: String, proposal_type: String },
  WorkspaceConflict { workspace_id: ComponentId, message: String },

  // Namespace errors
  NamespaceNotFound { path: String },
  NamespaceConflict { path: String, existing_id: ComponentId },
  InvalidNamespacePath { path: String, reason: String },
  FederationError   { peer_node: String, message: String },

  // Link Network errors
  LinkEdgeNotFound  { edge_id: EdgeId },
  ConsentRequired   { edge_type: LinkEdgeType, target_id: ComponentId },
  ShadowSyncFailed  { shadow_id: ShadowRowId, message: String },
  CircularLink      { from: ComponentId, to: ComponentId },

  // Identity errors
  PartitionNotFound { partition_id: UUID },
  CrossPartitionDenied { source_tag: IdentityTag, target_tag: IdentityTag },
  IsolationViolation { partition_id: UUID, attempted_access: String },

  // CRDT errors
  MergeConflict     { component_id: ComponentId, conflict_type: String },
  CrdtSyncFailed    { peer_node: String, message: String },
```

```
// Engine errors
WritebackDenied { engine: String, column_id: ColumnId, reason: String },
ComputationFailed { model: String, component_id: ComponentId, error: String },
}
```

16.2 Resilience Patterns

Pattern	Applied To	Implementation	Fallback Behavior
Circuit Breaker	kogi-engine calls from spreadsheet-service	Resilience4j (JVM) / failsafe-rs (Rust). Trip at 50% error rate over 10s.	Return last cached AIColumn value. Mark column as stale in UI.
Retry with backoff	Shadow row sync operations	Exponential backoff: 1s, 2s, 4s, 8s. Max 3 retries. After 3: shadow_status=Degraded.	Last successful sync values retained. User notified.
Read-your-writes	All component mutations	After write, route subsequent reads to the same PostgreSQL primary for 5s.	Consistency window. After 5s, reads from replica.
Optimistic Locking	Concurrent component updates	VectorClock comparison on write. Conflict → reject + return MergeConflict error.	Client receives conflict details; can retry with merged state.
Idempotency Keys	All POST operations (create, link, contribute)	X-Idempotency-Key header. Stored in Redis for 24h. Duplicate requests return cached response.	Prevents duplicate component creation / link creation.
Saga Pattern	Cross-service operations (escrow+deal+portfolio)	Each step has a compensating transaction. Saga coordinator in portfolio-service.	On failure: compensating transactions executed in reverse order.
Read Replicas	High-volume sheet queries (GET /sheets/:id/rows)	PostgreSQL streaming replication. Read replicas for search and analytics queries.	Replica lag <1s. On replica failure: fallback to primary.
Cache Stampede Prevention	Namespace resolution + Cell cache warm-up	Redis SETNX with soft TTL. One miss triggers recompute; others wait for result.	Prevents thundering herd on cold start or mass cache expiry.
Graceful Degradation	AI column unavailability (kogi-engine down)	Serve cached values with stale indicator. Sheet renders with partial data.	UI shows "AI features temporarily unavailable" banner. Core data always available.
Dead Letter Queue	Failed Kafka event processing	DLQ per Kafka consumer group. Failed events retried 3x, then parked in DLQ.	DLQ monitored. Manual replay after root cause fix.

16.3 Observability Stack

Signal Type	Tool	Key Metrics	Alert Conditions
Metrics	Prometheus + Grafana	Request rate, error rate, p50/p95/p99 latency per endpoint; CellStore hit rate; CRDT sync lag; Shadow sync queue depth; engine writeback queue depth	p99 > 500ms; error rate > 1%; CellStore miss > 20%; CRDT lag > 30s
Traces	OpenTelemetry + Jaeger	Distributed traces for all cross-service requests. Span tags: component_id, space_id, identity_tag, engine_name	Trace spans > 2s flagged for review
Logs	Structured JSON → Loki	All EventLog entries, CRDT operations, error events, authentication events, governance actions. Indexed by space_id, component_id	Error log rate spike; authentication failure burst
Audit	Immutable EventLog + CloudTrail	All component mutations, identity operations, Space governance actions, KLNK link events, engine writebacks	Cross-partition access without auth; Owner permission change; identity merge
Synthetic	k6 / synthetic monitors	End-to-end sheet load test every 5min; KLNK link creation test; Shadow sync latency probe; Namespace resolution latency probe	Sheet load > 3s; Link creation failure; Shadow sync > 30s

17. Performance & Scalability

17.1 Performance Targets

Operation	Target (p50)	Target (p99)	Current Bottleneck	Mitigation
Sheet query (100 rows)	<50ms	<200ms	JSONB deserialization + computed column eval	Computed column cache; result set compression; parallel column eval
Sheet query (1000 rows)	<200ms	<800ms	PostgreSQL sort + aggregation at scale	Materialized views for common aggregations; indexed sort columns
Namespace resolution	<5ms	<20ms	PostgreSQL lookup on cold cache	Redis cache (300s TTL); pre-warm on Space load
KLNK tree traversal (depth 3)	<100ms	<500ms	Recursive graph traversal in PostgreSQL	Adjacency list + closure table for common depths; pre-computed forests
ShadowRow sync (MirrorColumn)	<500ms	<2s	Kafka event → sync pipeline latency	Priority queue for active ShadowRows; WebSocket push for real-time
CRDT merge (100 ops)	<50ms	<200ms	VectorClock comparison + OR-Set merge	Rust native CRDT operations; batched merge for federation
Full-text search (10k component)	<50ms	<200ms	Meilisearch index lookup	Incremental index update; result caching for common queries
Engine writeback (single column)	<200ms	<1s	kogi-engine computation + network + PG write	Engine result batching; async write pipeline; CellStore pre-populate
Space health computation	<500ms	<3s	Recursive member + portfolio aggregation	Materialized Space health score; invalidate on Space events
WebSocket broadcast (100 clients)	<50ms	<200ms	Redis pub/sub fanout + WS write	Redis cluster; connection pooling; message batching

17.2 Scalability Architecture

Dimension	Strategy	Current Limits	Scale Path
Components per Entity	PostgreSQL HASH partitioned by owner_entity_id. Each partition independently queryable.	10M components per entity (estimated)	Shard by entity ID range; tiered storage for archived components
Spaces per Platform	Space metadata in separate PostgreSQL schema. Space ID in component table enables cross-Space queries.	1M+ active Spaces	Namespace-level sharding; read replicas per high-traffic Space

Dimension	Strategy	Current Limits	Scale Path
KLNK Edges (global)	graph_edges table with source/target indexes. SpaceLinkSubgraph materializes hot Space sub-graphs.	1B+ edges globally	Graph database (Neo4j) for global traversal; PG for local hot-path
Shadow Rows per User	shadow_rows table indexed by host_entity_id. Async sync pipeline scales horizontally.	10K ShadowRows per user (estimated)	ShadowRow sharding by host entity; Kafka partition by user ID for sync
Concurrent WS Sessions	Redis pub/sub for message fanout. Nginx WebSocket load balancing.	100K concurrent sessions	Horizontal scale of workspace-service; Redis Cluster for pub/sub
Namespace Registry Size	Redis cache + PostgreSQL with path index. Resolution <5ms at 100M+ entries.	1B+ namespace entries	B-tree index on path; Redis cache with LRU eviction; CDN-cached public paths
CRDT Operations/sec	Rust CRDT service with in-memory CrdtLog + async PostgreSQL flush.	100K ops/sec per node	Horizontal scale of crdt-service; Kafka for async persistence
Engine Writeback Volume	Batched writes via WritebackService. CellStore Redis absorbs burst writes.	1M writebacks/min	Write batching; Redis pipeline mode; async EventLog append

18. Oba AI Integration — Full Spreadsheet Layer

Oba is the AI chief-of-staff embedded in every sheet, every Workspace, and every Space. It operates in three modes: Reactive (answers questions and performs requested actions), Proactive (notifies what users should notice), and Autonomous (executes pre-approved workflows without per-action confirmation). All autonomous actions require explicit user enrollment and can be revoked at any time.

18.1 Oba Capabilities by Layer

Layer	Capability	Description	Trigger
Sheet	Row Annotations	Inline annotations on rows: health warnings, deadline alerts, budget risks, missing fields	Engine threshold crossing; field empty; staleness
Sheet	Smart Filters	Natural language → ViewFilter. "Show everything at risk this week" → PQL WHERE clause	Natural language query in search bar
Sheet	Formula Suggestions	Suggest derived columns based on existing column types. "Add profit margin %" when revenue+expenses detected.	New column of financial type created
Sheet	Batch Action Prep	"Archive all completed projects older than 90 days" → prepare action for confirmation	Natural language instruction
Sheet	Sheet Generation	"Create a sheet showing income by platform this quarter" → generate ViewDefinition + create SHT-034	Natural language view request
Sheet	Import Mapping	Suggest column mappings during CSV/XLSX import. Identity partition assignment prompt.	Import wizard launch
Sheet	Anomaly Alerts	Proactive flags: budget overrun approaching, velocity drop, stale project (>14 days), coverage gap, sync failure	AnomalyEngine + TelemetryEngine events
Workspace	Daily Briefing	Morning briefing card: health risks, deadlines, pending approvals, network changes, grant matches	Scheduled (morning) + on-demand
Workspace	Session Context	Oba remembers what the user was working on in the last session and offers to resume	WorkspaceSessionOpen
Workspace	Cross-Sheet Actions	"Move this task to the next sprint" → finds task row + sprint row → updates relationship across sheets	Natural language action
Space	Governance Drafting	Converts natural language to structured governance proposals. Quorum monitoring.	Space governance action

Layer	Capability	Description	Trigger
Space	Member Health Monitor	Identifies at-risk members (low activity, unreviewed contributions). Suggests steward outreach.	SpaceHealthEngine low score
Space	Cross-Space Match	"@partner-org has complementary skills to our project gap — draft collaboration inquiry?"	MatchEngine + KLNK mutual connection
Identity	Identity Audit	Review ProfilePartitions: incomplete profiles, stale views, PII exposed in public rows, untagged rows	Scheduled weekly; on partition update
Identity	Visibility Review	"Before publishing this, 3 columns will become visible to followers: bank_account_ref..." — confirm or adjust	Visibility change to Public/Protected
KLNK	Network Introductions	"You and @studio-collective have 8 mutual connections and shared interests — want an intro?"	KLNK mutual connection + MatchEngine score
KLNK	ShadowRow Health	"@partner ShadowRow hasn't synced in 72 hours. Review or refresh?"	Shadow sync failure; stale threshold
KLNK	Due Diligence Brief	"@contractor-x: 12 projects (avg health 84), 47 network connections, 4 in your network — full report?"	Deal room open; investment consideration
Namespace	Path Suggestions	Suggest namespace paths for new Spaces, portfolios, and projects based on existing hierarchy	New Space/portfolio creation wizard
Narrative	Portfolio Narrative	"Write a portfolio highlight for my top 3 Q1 projects" → structured narrative from row data for proposals/grants	User request; quarterly review reminder

18.2 Oba Workspace State Machine

```
pub struct ObaWorkspaceState {
    pub mode: ObaMode, // Reactive | Proactive | Autonomous
    pub active_context: ObaContext, // what Oba currently "knows" about the session
    pub pending_actions: Vec<ObaPendingAction>, // actions awaiting user confirmation
    pub briefing_generated: bool,
    pub last_anomaly_scan: DateTime<Utc>,
    pub enrolled_workflows: Vec<AutonomousWorkflowId>, // approved autonomous workflows
}

pub enum ObaMode {
```

```
    Reactive,    // responds to explicit user requests only
    Proactive,   // surfaces anomalies and suggestions unsolicited
    Autonomous { approved_action_types: Vec<ActionType> }, // executes approved
workflows
}

pub struct ObaPendingAction {
    pub action_id:    UUID,
    pub description:  String,           // human-readable description
    pub pql_preview: Option<String>,    // PQL or action spec for transparency
    pub affected_rows: Vec<ComponentId>,
    pub expires_at:   DateTime<Utc>,   // action expires if not confirmed within
window
    pub status:       PendingActionStatus, // AwaitingConfirmation | Confirmed |
Rejected | Expired
}
```

19. Platform Integration Map

kogi-* Module	KIMDSS Dependency	Data Flow	Space/Workspace Scope	KLNK Effect
kogi-home	Reads Master Registry for dashboard: active program count, health summary, wallet balance, upcoming deadlines	Read: portfolio → home dashboard	Reads from active Workspace context	None
kogi-office	Reads SHT-004 (Projects) + SHT-005 (Tasks) for boards/gantt; writes task status and sprint completions back	Read+Write: portfolio ↔ office; CRDT	Project Workspace auto-created for active projects	CrossSpaceProject edges for shared projects
kogi-bank	Reads SHT-009,015,016,017; writes payment events, benefit balances, ledger transactions	Read+Write: portfolio ↔ bank; owner-level only	Financial data never Space-scoped below owner level	InvestedIn edges on investment; Contracted on deal
kogi-marketplace	Reads SHT-026; writes investment/donation events; reads talent from SHT-006	Read+Write: portfolio ↔ marketplace	Community Space shows marketplace listings to members	Contracted/InvestedIn/Follows edges on transaction
kogi-community	Reads public rows for community feed; writes engagement events to analytics columns	Read+Write: portfolio ↔ community	Community Spaces host the social layer	Follows/Subscribe/Endorses edges on engagement
kogi-exchange	Reads SHT-017,027; writes deal execution events and allocation records	Read+Write: portfolio ↔ exchange	Organization Space hosts exchange activity	InvestedIn/Contracted on deal completion
kogi-profile	Profile rows are PortfolioComponents in SHT-025; linktree in Resource rows; accounts sync to identity columns	Read+Write: portfolio ↔ profile	Each ProfilePartition = one public Space namespace node	Identity KLNK node; all follows/subscriptions
kogi-engine	All sheets via EventLog; writes computed AIColumns via WritebackService	Read+Write: portfolio ↔ engine; all engine signals	SpaceHealthEngine reads Space component data	LinkNetworkEngine updates link columns
kogi-studio	Reads/writes Artifact rows; links design assets to parent Projects via Hierarchy edges	Read+Write: portfolio ↔ studio	Creative Space / Private Studio	ResourceShares edges for shared creative assets
kogi-dev	Full read/write via REST+gRPC; PortfolioPlugin for custom models; custom column types; KOGI-APPSTORE templates	Full bidirectional via kogi-dev API	Developer API key scoped to specific Spaces	Developer KLNK nodes for API-created links
kogi-gig	Reads/writes Gig rows (SHT-011); platform contribution tracking synced to SHT-015; earnings to SHT-009	Read+Write: portfolio ↔ gig	Personal Space / Gig Worker template Workspace	OrgMembership edges to gig platform Spaces
kogi-wms	WBS items (SHT-005) are dual-registered in KIMDSS and WMS; analytics unified; sprint boards read from KIMDSS	Read+Write: portfolio ↔ wms; shared component registry	Team Workspace shares WBS view with kogi-wms	CrossSpaceProject edges for multi-Space projects

kogi-* Module	KIMDSS Dependency	Data Flow	Space/Workspace Scope	KLNK Effect
kogi-rms	All resource types (SHT-006) are KIMDSS PortfolioComponents; KIMDSS is the master resource registry	Read+Write: portfolio ↔ rms; KIMDSS is authoritative	Resource Spaces for shared tool/compute pools	ResourceShares edges for all shared resources
KOGI-MANAGER	Reads PolicyEngine; writes RBAC policies; enforces PermissionTier; manages governance config for Spaces	Policy enforcement: KOGI-MANAGER → portfolio	Space governance configs managed here	None (governance layer, not economic)
KOGI-APPSTORE	Distributes Portfolio Templates, custom column sets, computed plugins, view themes, Workspace templates, Space templates	Distribution: APPSTORE → portfolio templates	Template Spaces maintained by APPSTORE	CampusLink edges for template-to-user connections

20. Open Items, Decisions & Development Roadmap

20.1 Open Technical Decisions

Decision	Options	Current Leaning	Decision By
Status CRDT merge strategy	LWW Custom lattice Application-layer merge Conflict-surfacing only	Custom lattice (\$4.5) — preserves lifecycle semantics	v3.0 milestone
KLNK global graph store	PostgreSQL adjacency table Neo4j DGraph TigerGraph	PostgreSQL for hot path + Neo4j for global traversal (Phase 4)	v3.5 milestone
Namespace resolution performance	Redis cache CDN-cached public paths Materialized PG view All three	All three in tiered priority: Redis → PG MV → CDN	v3.0 milestone
Space CRDT federation	Per-Space federation Platform-wide federation Opt-in Space federation	Opt-in Space federation (federating specific Spaces, not all)	v3.5 milestone
HardIsolated partition storage	Separate PG schema Encrypted columns in shared table Separate PG instance	Encrypted columns with per-partition key (practical balance)	v3.0 milestone
ShadowRow update granularity	Full row refresh Column-delta push OR-Set delta	Column-delta push via Kafka (minimal data transfer)	v3.0 milestone
Formula Engine sandboxing	WASM sandbox Process isolation JVM sandbox Interpreted AST	Interpreted AST with resource limits (no arbitrary code execution)	v3.0 milestone
PQL implementation backend	Custom parser + PG query generator Presto/Trino Apache Calcite	Custom parser + PG query generator (tight schema integration)	v3.0 milestone
AI Autonomous mode permissions	Per-action type whitelist Budget-capped autonomous Time-limited autonomy All three	Per-action type whitelist + budget cap (belt and suspenders)	v3.5 milestone

20.2 Open Items — Current Sprint

Item	Prior ity	Description	Blocks
Persistence Layer Pluggability	P0	PortfolioStore trait abstraction for pluggable PostgreSQL, SQLite (local), and CouchDB (P2P) backends.	Multi-node deployment; offline mode
Status CRDT Lattice	P0	Implement the custom lifecycle lattice for status merges. Currently status ops are no-ops in CRDT.	Multi-node deployment; federated Spaces

Item	Priority	Description	Blocks
Space CRDT Sync	P0	Extend CRDT federation to Space-scoped components. Space members on different nodes see consistent state.	Organization Space multi-node
Cross-Partition SplitPolicy Enforcement	P0	SplitPolicy defined but not enforced at Substrate layer for cross-partition CrdtOperations.	Multi-identity deployments
Namespace Service Production-Ready	P1	NamespaceRegistry full implementation: alias management, federation sync, cache warm-up, health reporting.	Space discovery; KLNK traversal
KLNK Consent Flow UI	P1	The consent workflow for Collaborates/InvestedIn edge types (invitation, column visibility config).	Cross-portfolio collaboration
ShadowRow Real-Time Delta Sync	P1	Kafka consumer for shadow sync; column-delta push instead of full row refresh. Performance critical.	Cross-portfolio visibility at scale
EventLog Data Lake Flush	P1	10K event cap risks history loss. Plugin hook to flush to ClickHouse/S3 before eviction.	Long-lived component audit trail
WritebackService Permission Model	P1	Harden AI writer permission checks. Currently trust-based; needs cryptographic engine identity.	Production kogi-engine integration
Formula Engine v1	P1	Full ColumnComputer expression language evaluator with all listed functions including RELATED() and engine functions.	Custom column power users
PQL Full Implementation	P2	Parser + evaluator with JOIN support, SHADOW INCLUDE, AS_OF time-travel, UNDER namespace filter.	Advanced sheet queries
Real-Time Collaboration UI	P2	Live cursor indicators, cell lock indicators, presence badges in sheet header.	Shared Workspace experience
Space Link Subgraph Materialization	P2	Hourly materialized view of SpaceLinkSubgraph per Space. Currently computed on demand (slow for large Spaces).	Space link visualization performance
Identity Merge Conflict Wizard	P2	UI for CrossIdentityMerge with conflict detection and manual resolution for shared component edge cases.	Multi-identity power users
Template Marketplace (APPSTORE)	P3	KOGI-APPSTORE integration for Space templates, Workspace templates, column sets, and view themes.	Template ecosystem
Federated Namespace Sync	P3	Replicate namespace entries to federation peers. Currently federation peers cannot resolve each other's paths.	Cross-federation path resolution
On-Chain Identity Anchoring	P3	DID (W3C Decentralized Identifier) anchoring for identity handles. Cross-platform reputation portability.	Web3 identity federation

20.3 Version Roadmap

Version	Milestone	Key Deliverables
v2.2 (Current)	Foundation	Master Registry · Hierarchy · Projects · Finances · Benefits · Work & Gigs · PortfolioHealth v1 · Basic CRDT · PostgreSQL persistence · Single-identity · Basic Space support
v2.5	Intelligence + Collaboration	AI ComputedColumns · CollaborationEngine · Shared Portfolios · Contribution Attribution · CrowdfundingCampaign · Grant & Crowdfunding Sheets · WritebackService v1 · Space governance v1
v3.0	Full Spreadsheet Platform	All 38 Sheets · PQL v1 · Formula Engine v1 · All Board Modes · Real-time Collaboration UI · Full Import/Export · Oba Spreadsheet Intelligence · KLNK v1 · Multi-identity KPID v1 · Namespace v1 · Space v1 full implementation · Workspace templates
v3.5	Link Network + Multi-Identity + Spaces	Full KLNK graph API · LinkForest/Tree rendering · KLNK consent flow · ShadowRow real-time delta sync · Space CRDT sync · CrossIdentityMerge UI · IdentityCoverage score · SpaceHealth engine · Workspace session history · Namespace federation sync · APPSTORE template marketplace
v4.0	Autonomous Intelligence + Federation	Oba autonomous mode (approval-first) · Predictive scenario simulation · AI-generated portfolio narratives · Cross-platform reputation portability · DID anchoring · Federated MatchEngine across ShangoOS · Neo4j global KLNK graph · Space federation (opt-in)
v4.5	Web3 + Decentralization	On-chain identity anchoring · Decentralized portfolio storage (IPFS + CouchDB) · Smart contract cooperative distribution · On-chain equity/cap table · Cross-federation KLNK traversal · DAO governance integration · Real-world asset tokenization for cooperative Spaces

20.4 Glossary

Term	Definition
KIMDSS	Kogi Interconnected Master Distributed Spreadsheet System — the complete system described in this document
KSPC	Space module — named, governed, bounded digital environments (Personal, Team, Org, Coop, Community, Federation, Event, etc.)
KWSP	Workspace module — active working context within a Space: open sheets, session state, tools, presence
KNSPC	Namespace module — hierarchical addressing for all platform entities via NamespacePath URIs
KPMS	Portfolio Management System — master orchestration layer
KPSS	Portfolio Spreadsheet Substrate — Rust low-level data engine

Term	Definition
KPRG	Portfolio Component Registry — canonical index of all PortfolioComponents
KPVW	View Engine — filter/sort/group/pivot/board mode transforms
KPCM	Computation Engine — derived columns and analytical models
KPCL	Collaborative Editing — CRDT-backed multi-user real-time editing
KPEX	Export & Integration — CSV/XLSX/JSON/webhook/embed
KLNK	Link Network — inter-portfolio graph: forests, trees, cross-user connections
KPID	Portfolio Identity System — multi-account, multi-identity, multi-profile management
Space	A named, governed, bounded PortfolioComponent of ContainerType::Space. Top-level organizational context.
Workspace	Active working context within a Space. Personalized sheet arrangement, session state. ContainerType::Workspace.
NamespacePath	A hierarchical URI addressing a Kogi entity: kogi://{scope}/{type}/{slug}/...
NamespaceRegistry	The canonical database mapping NamespacePaths to ComponentIds. The platform's DNS equivalent.
Sovereign Entity	The real-world person, org, or collective that owns one root PortfolioSystem instance and all associated identities
ProfilePartition	A logical partition of the root spreadsheet's rows tagged to one Identity. Not a physical copy.
VisibilityMask	Rules specifying which rows and columns are visible to which observer type (Public/Follower/Connection/Owner)
SplitPolicy	Governance policy on cross-partition data access, enforced by PolicyEngine
InterPortfolioLink	A GraphEdge crossing the boundary between two different users' root spreadsheets. The KLNK atom.
LinkEdge	A typed, directional, weighted edge between two LinkNodes in the KLNK graph
LinkForest	All LinkTrees rooted at a given Sovereign Entity — the complete view of their network
LinkTree	A rooted directed subgraph of KLNK from a single root node to a configured max depth
SpaceLinkSubgraph	The materialized sub-graph of KLNK scoped to a single Space and its members
ShadowRow	A read-only row in User B's spreadsheet mirroring a linked component from User A's spreadsheet
MirrorColumn	A specific column from a ShadowRow synced into the host spreadsheet as a read-only ComputedColumn
VectorClock	HashMap<NodeId, u64>. Logical clock for causal ordering of concurrent mutations across federation nodes
CrdtLog	LWW + OR-Set operation log. Applied on merge to resolve concurrent edits without data loss.
PortfolioRow	A PortfolioComponent projected as a spreadsheet row: flat key-value map of ColumnId → TypedCellValue

Term	Definition
SheetDefinition	A named, scoped, filterable view: base row type filter + visible column schema + default sort/group
ViewDefinition	User-saved customization of a SheetDefinition: filters, sorts, groups, highlight rules, board config
ComputedColumn	A derived column computed from other columns. Tier 1 = sync arithmetic; Tier 2 = async AI engine signal
AIColumn	Engine-generated signal written by kogi-engine via WritebackService. Read-only to users.
WritebackService	gRPC service accepting AI-computed values from kogi-engine, validating permissions, writing to CellStore
PQL	Portfolio Query Language — SQL-inspired query language typed to the PortfolioRow schema
EventLog	Append-only, immutable log of every mutation on a PortfolioComponent. Never deleted.
Oba	The Kogi platform AI assistant — Reactive, Proactive, and Autonomous modes across all KIMDSS layers
kogi-engine	The Scala 3 intelligence engine consuming all portfolio events and writing back computed signals via WritebackService
CRDT	Conflict-free Replicated Data Type — LWW + OR-Set + Status Lattice — enables safe concurrent edits across all nodes